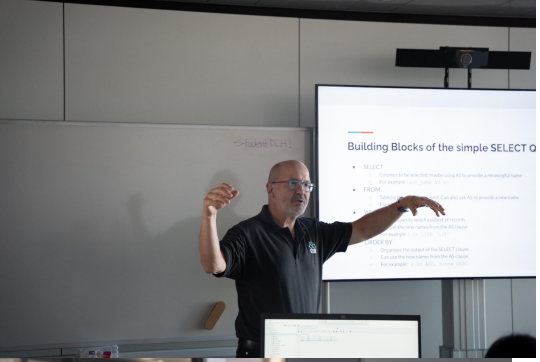
A hands-on introduction to Relational Database and SQL with Postgres

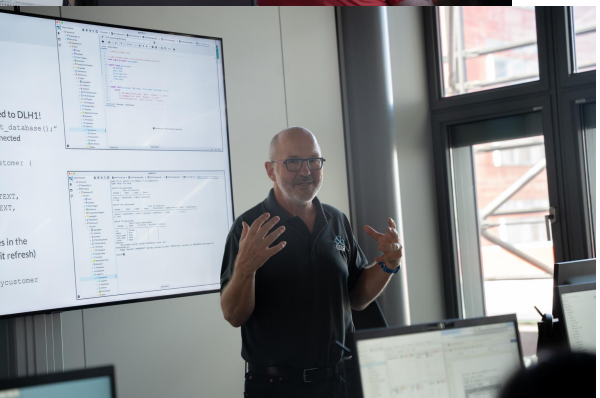
Marc Linster

September 2026

© 2025 Marc Linster - All rights reserved





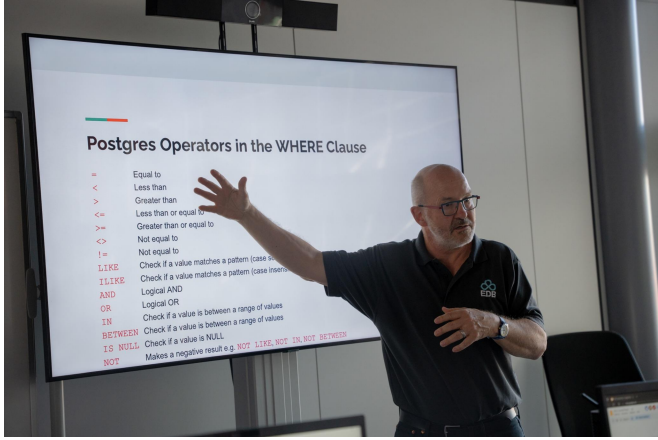


Marc Linster, Ph.D.

Former CTO of EDB, largest contributor to Postgres

Co-organizer of Postgres Europe, largest Postgres conference in the world

marc@marclinster.com
https://www.linkedin.com/in/marclinster;
https://github.com/marclinster





Lesson Plan

- **14:00 - 17:00**
- **Start with review of prior day**
- **End with time to try things out and ask questions**
- **Hands on**

Monday	• Quick intro to databases • Tables and queries:Basics • Query basics: Insert, update, delete, select • *Time to exercise and ask questions*
Tuesday	• *Review of prior session and Q&A* • Operators in the WHERE clause • Calculations in the SELECT clause • Primary Keys, Constraints and Relationships • *Time to exercise and ask questions*
Wednesday	• *Review of prior session and Q&A* • Organizing data with schemas and entity-relationship diagrams • Data Normalization • Queries revisited: Joins • Views
Thursday	• Queries revisited: aggregates • Indexes • Writing good queries • A simple project • Recap • Resources

Intro, Advanced, and AI Classes

Practical Introduction to SQL with Postgres

Audience: Beginners
Tool: Graphical UI (pgAdmin)
Duration: 4 days @ 3 hours
Goals:

- Familiarize with SQL and relational databases
- Write simple queries
- Read and understand data models

Advanced SQL with Postgres

Audience: Prior hands-on experience with database or programming tools
Tool: Command line (psql) & GUI (pgAdmin)
Duration: 5 days @ 3 hours
Goals:

- Read and write complex queries
- Understand transactions, indexes, and analytics concepts
- Develop normalized data models
- Develop stored procedures

Postgres and AI

Audience: Advanced users with SQL, stored procedures and Python knowledge
Tool: Command line (psql) & VStudio
Duration: 5 days @ 3 hours
Goals:

- Understand AI concepts
- Understand vectors and semantic search
- Learn integration with LLM
- Develop chat bot

Day 1

- Connect to the VMs
- Intro to databases
- Connecting to a database
- Tables and queries: Basics
- Query basics: Insert, update, delete, select

Connecting to the DLH Course VMs

Quick Intro to Databases

What is a database?

A **database** is an organized collection of structured data, typically stored electronically in a computer system.

Use Case Examples

- All orders that were taken, shipped, invoiced and paid in a store or eCommerce applications
- List of all laws that are currently in place
- Register of all the inhabitants of a country with their age, marital status, and domicile
- Collections of documents, X-ray images, chat conversations, and emails
- The WWW
- Data can include numbers, text, geo-locations, images, videos, links, vectors, lists, …

Key Functions of a Database

- **Data Storage**: Securely store large volumes of data.
- **Data Retrieval**: Quickly fetch specific data using queries.
- **Data Manipulation**: Insert, update, or delete data as needed.
- **Data Integrity**: Ensure accuracy and consistency through constraints and rules.
- **Data Security**: Protect data from unauthorized access with authentication and encryption.
- **Data Analysis**: Support complex queries and reporting for decision-making.

Components of a Database

- **Data**: The actual information stored, such as customer records, product details, or transaction logs.
- **Database Management System (DBMS)**: Software that interacts with the database, enabling users to define, create, query, update, and manage data. Examples include MySQL, PostgreSQL, Oracle, and MongoDB.
- **Schema**: The structure or blueprint of the database, defining how data is organized (e.g., tables, fields, relationships).
- **Query Language**: A language used to interact with the database, such as SQL (Structured Query Language) for relational databases.

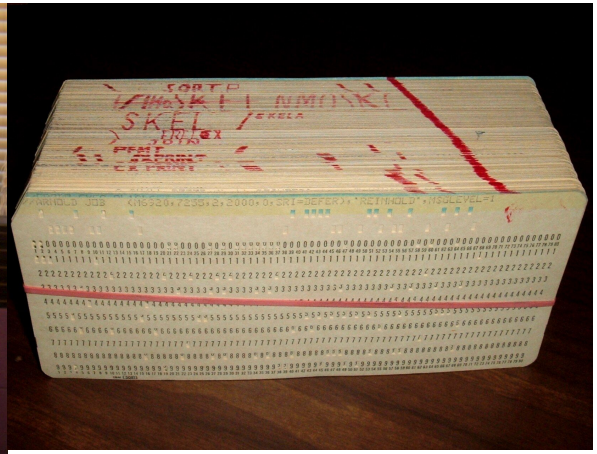
How to organize data so that machines can understand it - an age old problem



Jacquard Loom - 1801



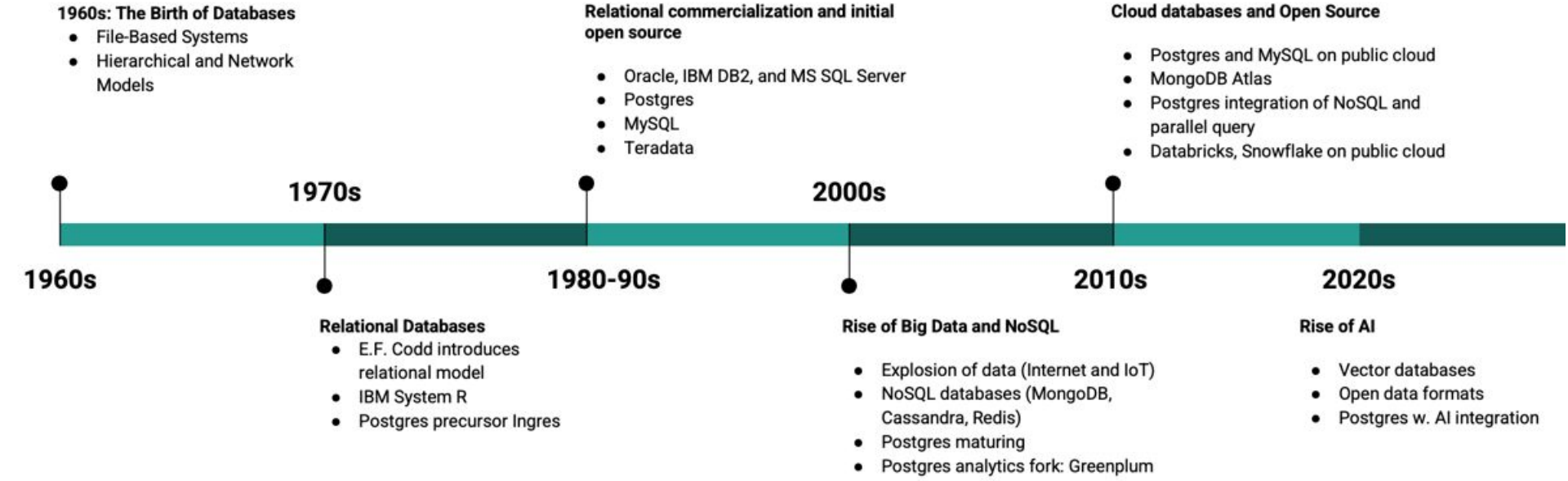
Hollerith Census Counter - 1890 (later: IBM)



Punch cards (IBM PL/1 - 1960)

Physically stored!

History of Databases

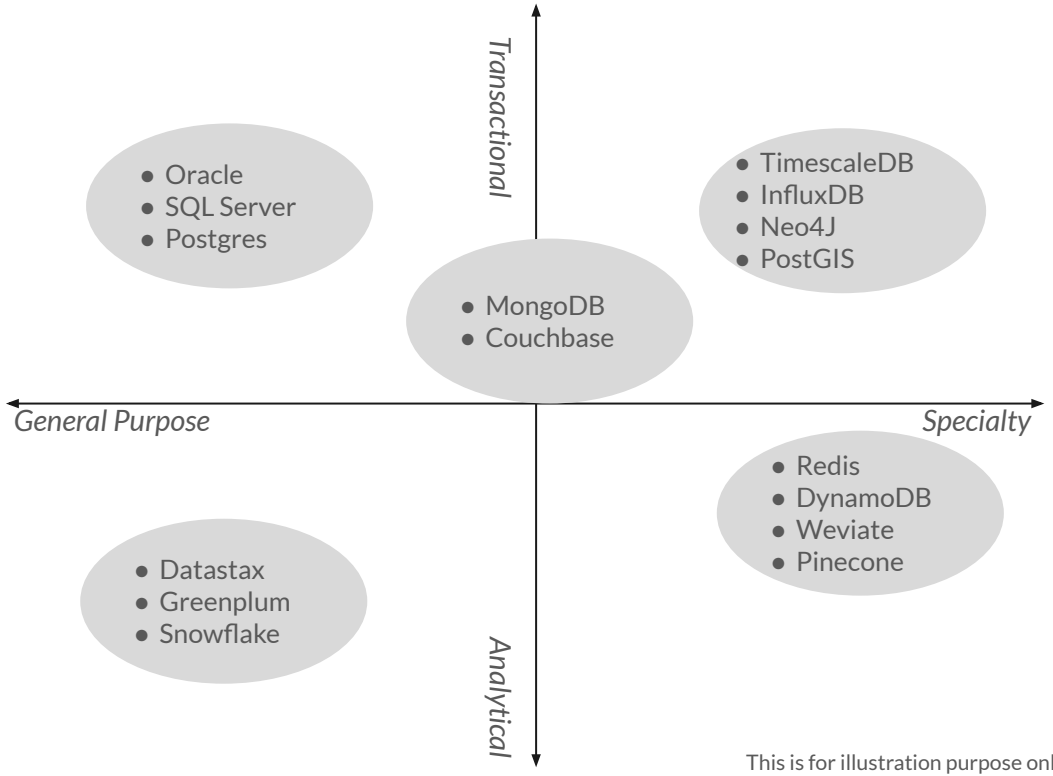



30
YEARS OF POSTGRESQL

Principal Types of Databases

- There are many, many types of databases
- Key database types (not exhaustive)
  - Relational ⇐ Our focus this week (Oracle, My SQL, Postgres, SQL Server, IBM DB2, Sybase, …)
  - Document (MongoDB, Couchbase, …)
  - Key Value Pair (Redis, RocksDB, Amazon Dynamo, …)
  - Graph (Neo4J)
  - AI and vector (Pinecone, Weviate, …)
- Features are merging!
  - Relational databases are adopting graph, document, KVP, and AI vector capabilities

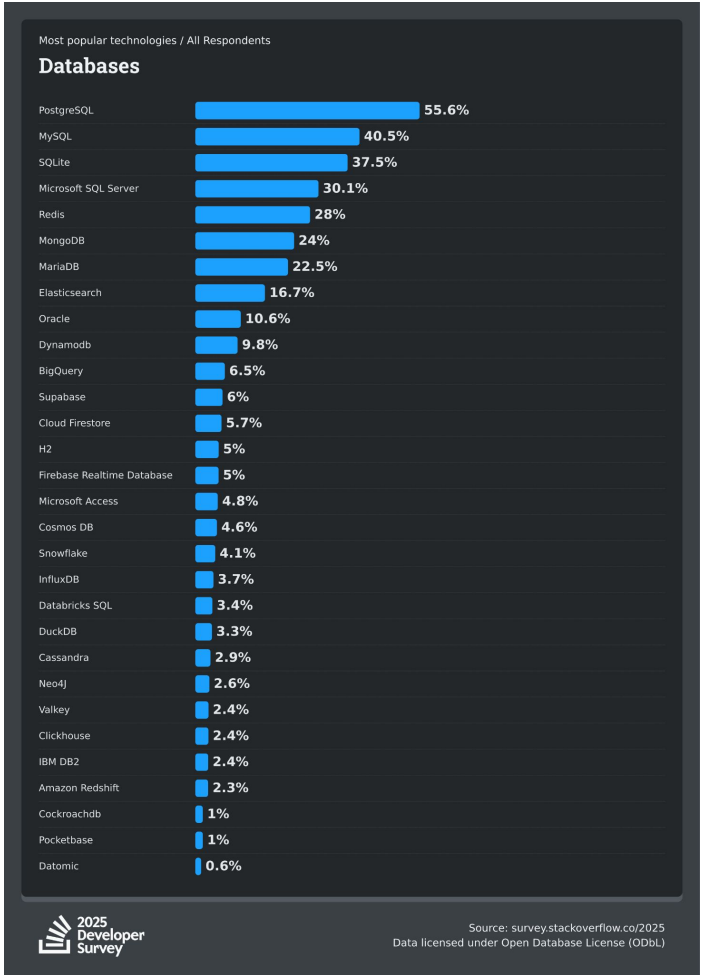
Database Landscape



This is for illustration purpose only
There are many, many databases and many ways to classify them

Stackoverflow Developer Survey 2025

Year	Rank	Developers using Postgres	Key Competitors
2025	#1	55.6%	MySQL (40.5%), SQLite (37.5%)
2024	#1	51%	MySQL (45%), SQLite (34%)
2023	#1	47%	MySQL (47%), SQLite (32%)
2022	#2	46%	MySQL (47%), SQLite (32%)
2021	#2	40%	MySQL (51%), SQLite (32%)
2020	#2	38%	MySQL (55%), SQLite (32%)
2019	#2	34%	MySQL (58%), SQLite (32%)
2018	#3	32%	MySQL (58%), SQL Server (36%)



Relational Databases

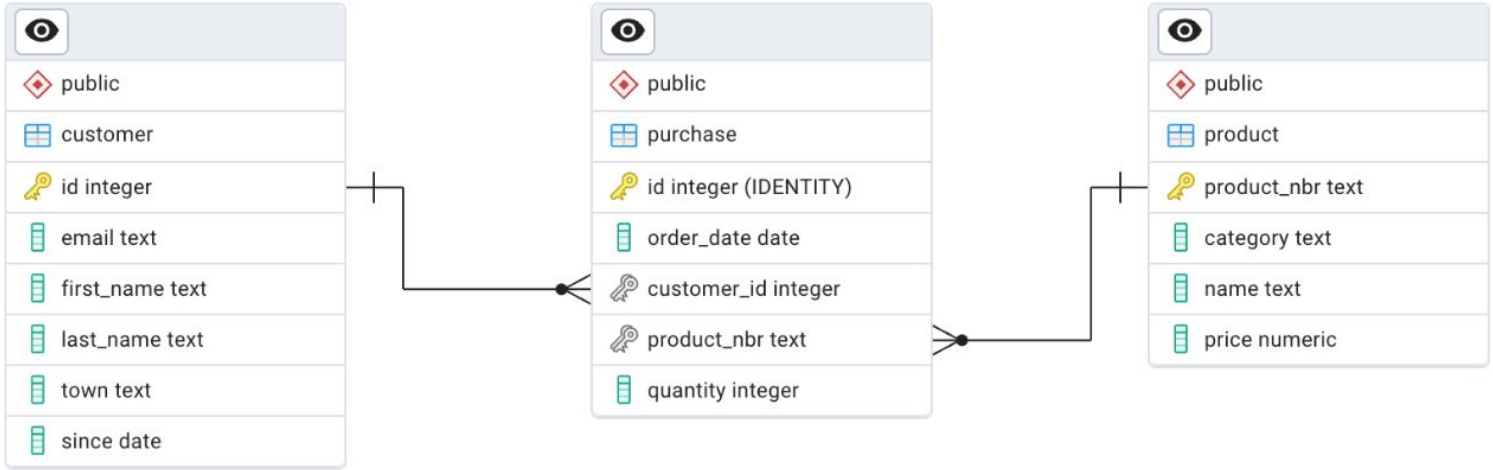
- Tables, e.g., products, customers
- Relations, e.g., which customer bought which product
- Schemas, e.g., structured collections of tables and relations
- Data types, e.g., integer, numeric, float, text, time, date, …
- Data Definition Language (DDL), e.g. CREATE TABLE, CREATE INDEX, ….
- Data Manipulation Language (DML), e.g., SELECT name FROM product, INSERT (first_name, last_name) VALUES (‘marc’, ‘linster’) INTO customer
- Data Control Language (DCL), e.g., GRANT SELECT ON product
- Transaction Control Language (TCL), e.g., BEGIN TRANSACTION, COMMIT, ROLLBACK

Relational Databases

id	timestamp	product_id	customer_id	store_id	qty	unit_price	extended_price
80	2025-12-26 6:41:08	1405	279	1	61	208.23	12702.03
8	2025-12-07 7:20:22	392	75	4	71	203.4	14441.4
61	2025-12-08 22:50:43	1476	237	2	27	155.91	4209.57
59	2025-12-17 11:45:05	2232	331	3	6	132.73	796.38
10	2025-12-11 19:56:24	113	33	4	23	153.06	3520.38
66	2025-12-22 9:03:43	2178	70	1	61	125.83	7675.63
51	2025-12-07 21:30:30	2082	485	2	61	138.98	8477.78
52	2025-12-03 16:47:22	1105	443	3	69	49.7	3429.3
79	2025-12-19 21:46:07	236	126	4	15	81.04	1215.6
36	2025-12-27 10:37:25	1359	469	1	59	95.92	5659.28

name	description	manufacturer_id	size	color	sku	price
Cotton T-Shirt	Handcrafted details	3	XXL	Light Green	FSH-89166-000000000165	11.4
Wool Socks	Trendy and fashionable	1	M	Dark Yellow	FSH-53451-000000000138	146.26
Wool Dress Pants	Elegant and stylish	2	S	Turquoise	FSH-25089-0000000001213	102.79
Evening Gown	Waterproof and windproof	3	XS	Light Green	FSH-47771-000000000115	103.36
Wool Dress Pants	Elegant and stylish	2	XXL	Dark Blue	FSH-13141-000000000164	16.41
Baseball Cap	Versatile and functional	3	XL	Light Red	FSH-43094-000000000151	160.5
Slacks	Trendy and fashionable	3	XL	Dark Blue	FSH-82072-000000000154	39.63
Leather Jacket	Comfortable and durable	3	XS	Amber	FSH-15582-0000000001114	172.11
Sunglasses	Quick-drying	2	L	Dark Blue	FSH-65791-000000000144	18.73
Leather Belt	Comfortable and durable	3	XXL	Amber	FSH-11391-0000000001614	62.81

Entity-Relationship Diagrams



Document Database

- Documents encapsulate and encode data, e.g., customer
- Collections gather similar documents, e.g., customer**s**
- No standard schema, e.g., some customer documents can have fields that differ. For example
  - Customer1: firstname, lastname, address_line_1, city, postalcode
  - Customer2: firstname, **middlename**, lastname, address_line_1, **address_line_2**, city, postalcode

Document Database


```
{
  "customer_id": "12345",
  "name": "John Doe",
  "email": "johndoe@example.com",
  "phone": "+1234567890",
  "address": {
      "street": "123 Main St",
      "city": "Anytown",
      "state": "CA",
      "zip": "90210"
  },
"orders": [
    {
      "order_id": "67890",
      “customer_id”: “12345”,
      "date": "2023-10-01",
      "total_amount": 150.75,
      "items": [
        {
            "product_id": "p123",
            "product_name": "Wireless
Mouse",
            "quantity": 1,
            "price": 50.25
        },
        {
            "product_id": "p456",
            "product_name": "Bluetooth
Keyboard",
            "quantity": 1,
            "price": 100.50
        }
      ]
    },
```

Key Value Pair Database

- ‘Values’ associated with a ‘Key’
  - `Customer: 123 name “Marc Linster” city “Junglinster”`
  - `Product:xyz price “192,17” color”blue”`

- Extremely fast access to data
- Usually stored in memory
- Used as cache or working memory for applications

Key Value Pair Database

```
HSET customer:12345 name "John Doe" email "johndoe@example.com" phone "+1234567890"
HSET customer:12345 address '{"street": "123 Main St", "city": "Anytown", "state": "CA", "zip": "90210"}'

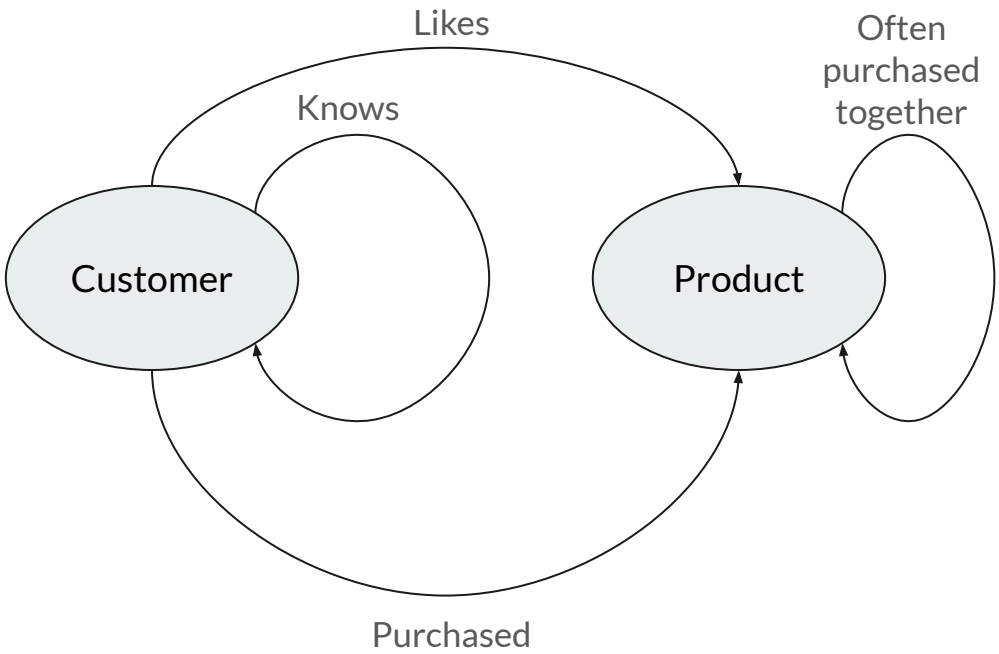
HSET customer:67890 name "Jane Smith" email "janesmith@example.com" phone "+0987654321"
HSET customer:67890 address '{"street": "456 Elm St", "city": "Othertown", "state": "NY", "zip": "10001"}'
```

```
HSET order:67890 customer_id "12345" date "2023-10-01" total_amount "150.75"
HSET order:67890 items '[{"product_id": "p123", "product_name": "Wireless Mouse", "quantity": 1, "price":
50.25}, {"product_id": "p456", "product_name": "Bluetooth Keyboard", "quantity": 1, "price": 100.50}]'

HSET order:67891 customer_id "12345" date "2023-10-05" total_amount "75.00"
HSET order:67891 items '[{"product_id": "p789", "product_name": "USB-C Hub", "quantity": 2, "price":
37.50}]'
```

Graph Database

- Nodes, e.g., customer, product
- Relationships, e.g., purchased, likes, knows



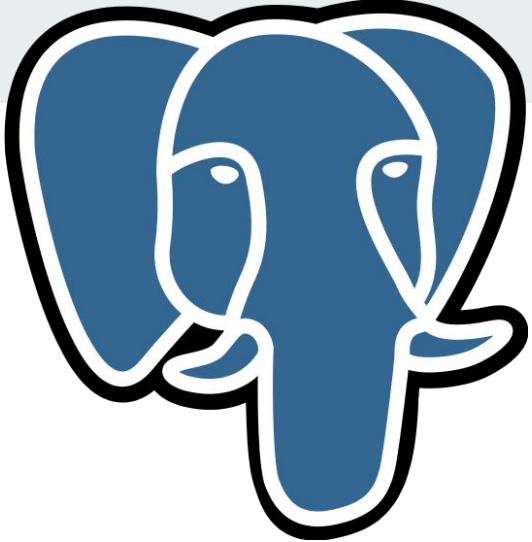
Graph Database

```
CREATE (c1:Customer {id: 1, name: "John Doe", email: "johndoe@example.com"});
CREATE (c2:Customer {id: 2, name: "Jane Smith", email: "janesmith@example.com"});
CREATE (c3:Customer {id: 3, name: "Alice Johnson", email: "alicej@example.com"});

CREATE (p1:Product {id: 101, name: "Wireless Mouse", price: 25.99});
CREATE (p2:Product {id: 102, name: "Bluetooth Keyboard", price: 49.99});
CREATE (p3:Product {id: 103, name: "USB-C Hub", price: 35.50});
CREATE (p4:Product {id: 104, name: "Monitor", price: 199.99});


MATCH (c1:Customer {id: 1}), (p1:Product {id: 101}) CREATE (c1)-[:LIKES]->(p1);
MATCH (c1:Customer {id: 1}), (p2:Product {id: 102}) CREATE (c1)-[:LIKES]->(p2);
MATCH (c2:Customer {id: 2}), (p3:Product {id: 103}) CREATE (c2)-[:LIKES]->(p3);
MATCH (c3:Customer {id: 3}), (p4:Product {id: 104}) CREATE (c3)-[:LIKES]->(p4);
```

Connecting to a Postgres Database



Tools for this course

Editor

- pgAdmin 9.15
- License: Postgres License

Postgres

- PostgreSQL V18.4
- License: Postgres License

Exercises and Samples

- License: Creative Commons (no Rights Reserved)
- Download at https://github.com/marclinster

It's all 100% Open Source

Take it home, use it,
learn from it

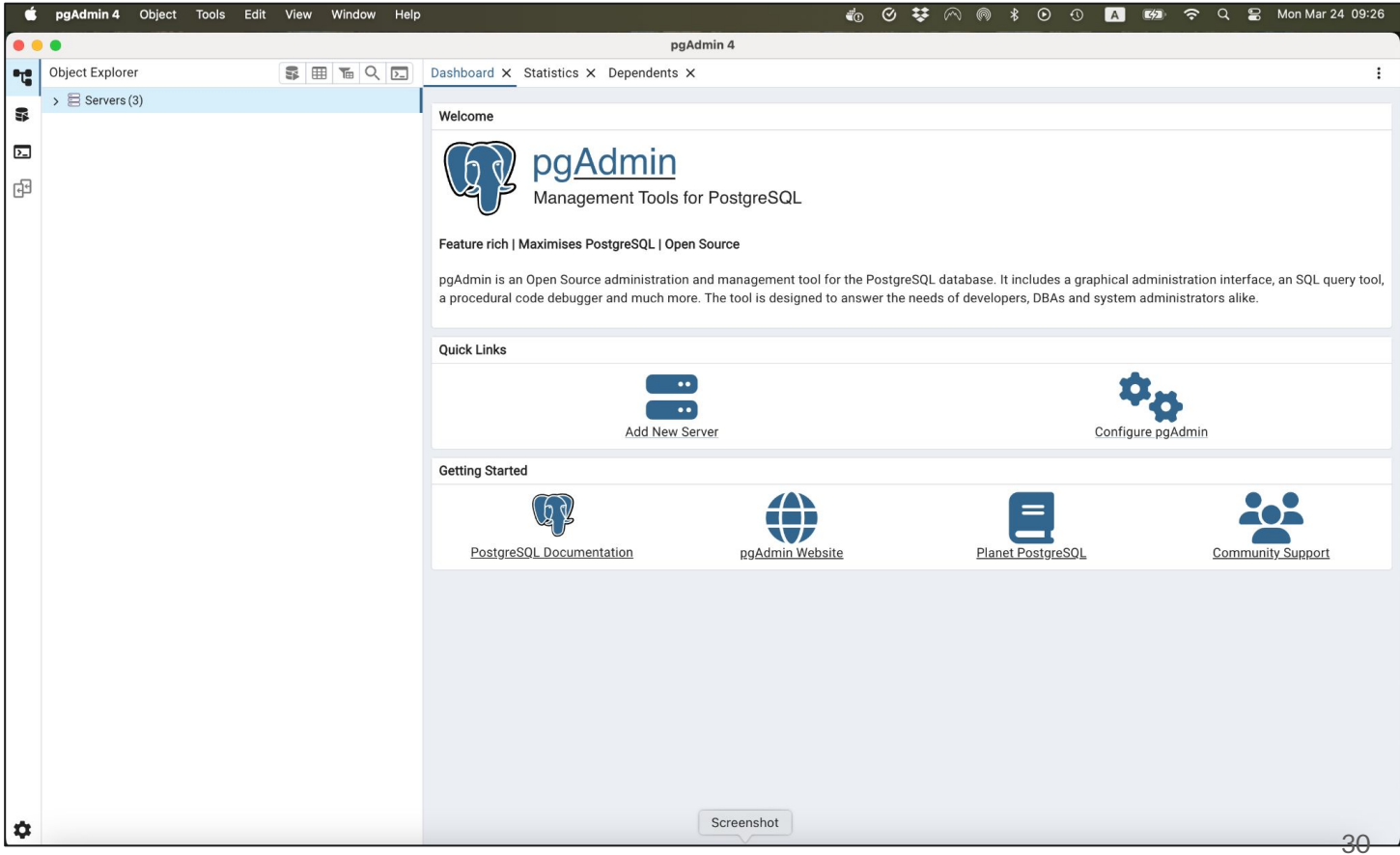
Basic elements of a Postgres database connection from pgAdmin

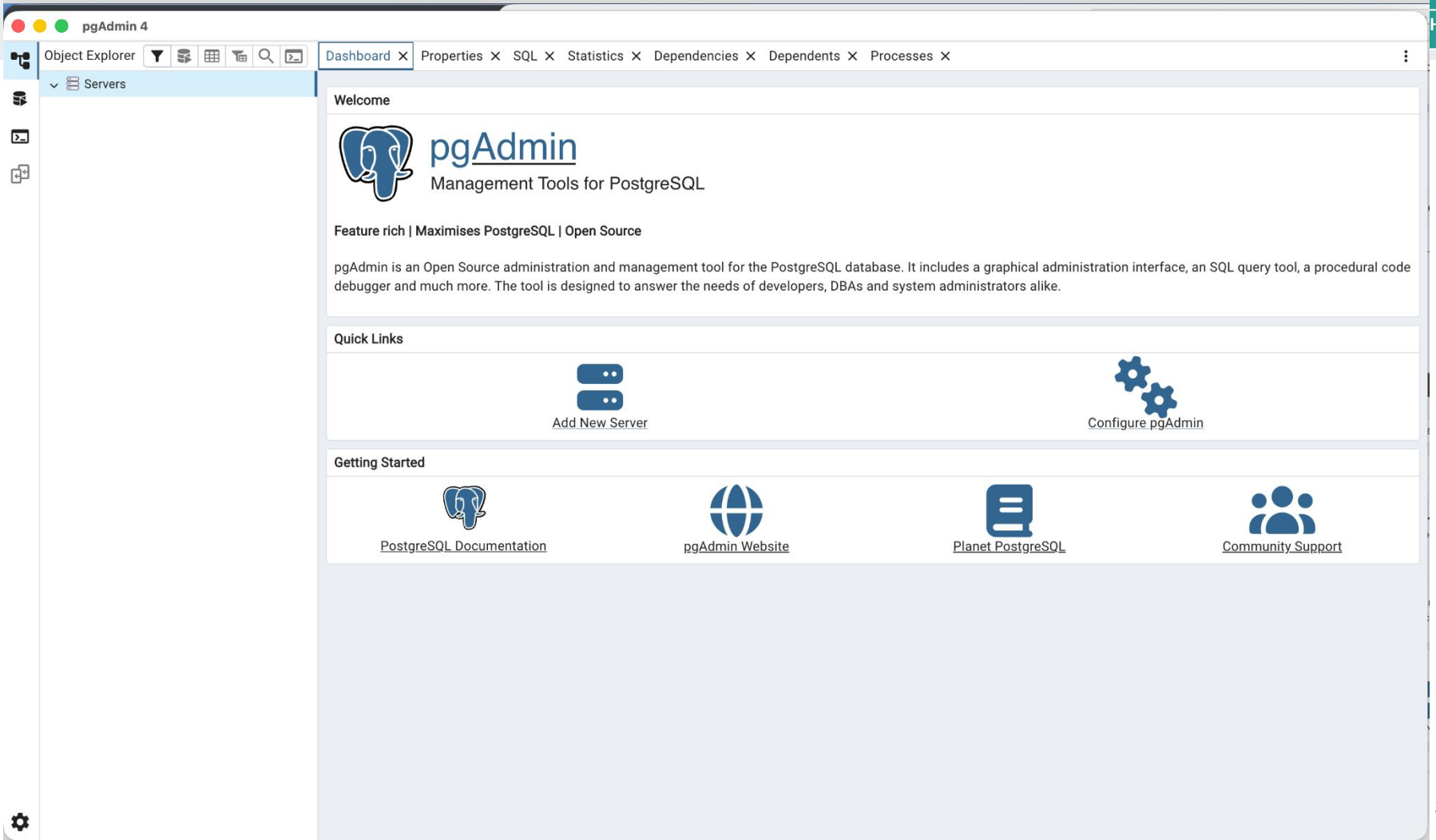
Connection name	The name in the GUI	• **Postgres18** ⇐ • My connection 1
Where is the database server?	Network address (IP address and port)	IP Address • **Localhost** ⇐ • 188.115.7.175 • mydb.aws.com Port • **5432** ⇐ • 80
Which database do I connect to?	Name of the database on the servers	• **dlh1** ⇐ • dlh2 • dlh3 • inventory
Who am I?	Username and password	Username • **postgres** ⇐ • dhlstudent Password • **postgres** ⇐ • xd67$566
Security parameters	Encryption on the network	• Not applicable in training

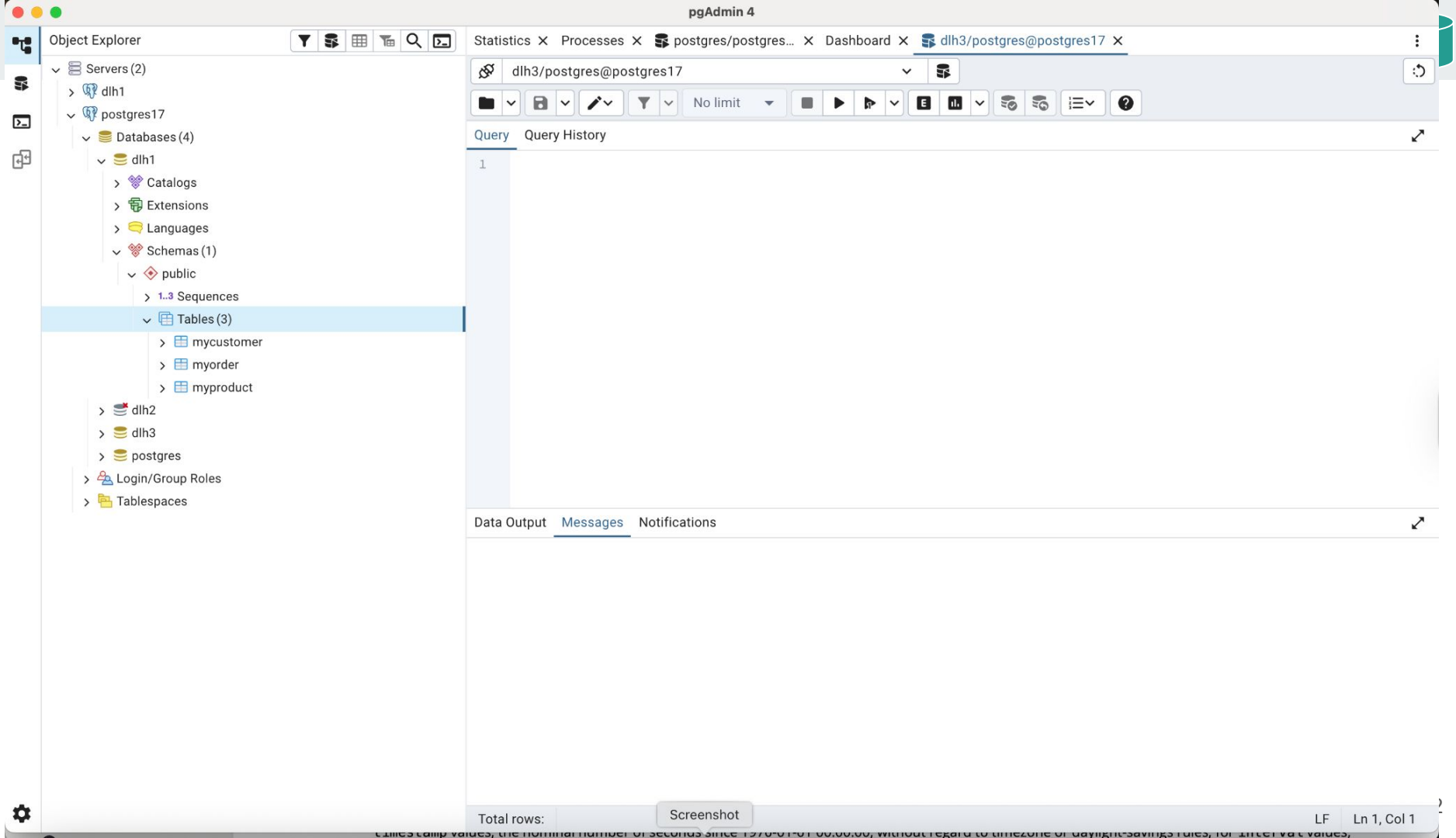
Connection Elements

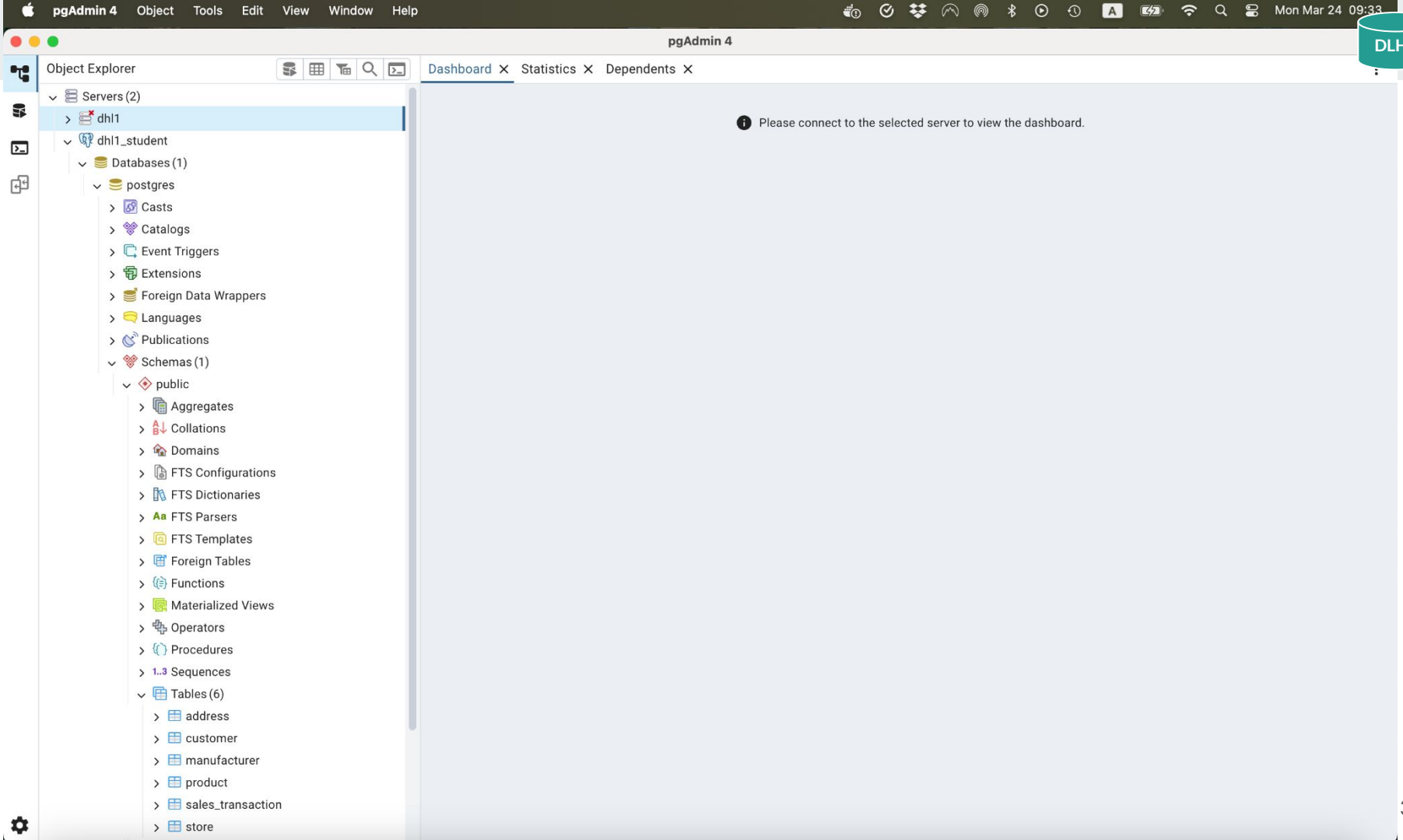
- Network address
  - IP V4 example: 188.115.7.175
  - IP V6 example: 2001:7e8:c415:f601:c0fe:c98b:257a:a45b
  - url: mydb.post.lu
  - Special address: **localhost** (always the local computer. In IPV4: "127.0.0.1", in IPV6: "::1")
- (Network) Port:
  - The port defines which service at a specific network address I want to use, and what 'language' (protocol) I can speak at that port
  - Examples:
    - Port 20: SMTP (email)
    - Port 80: HTTP (WWW)
    - **Port 5432: Postgres**

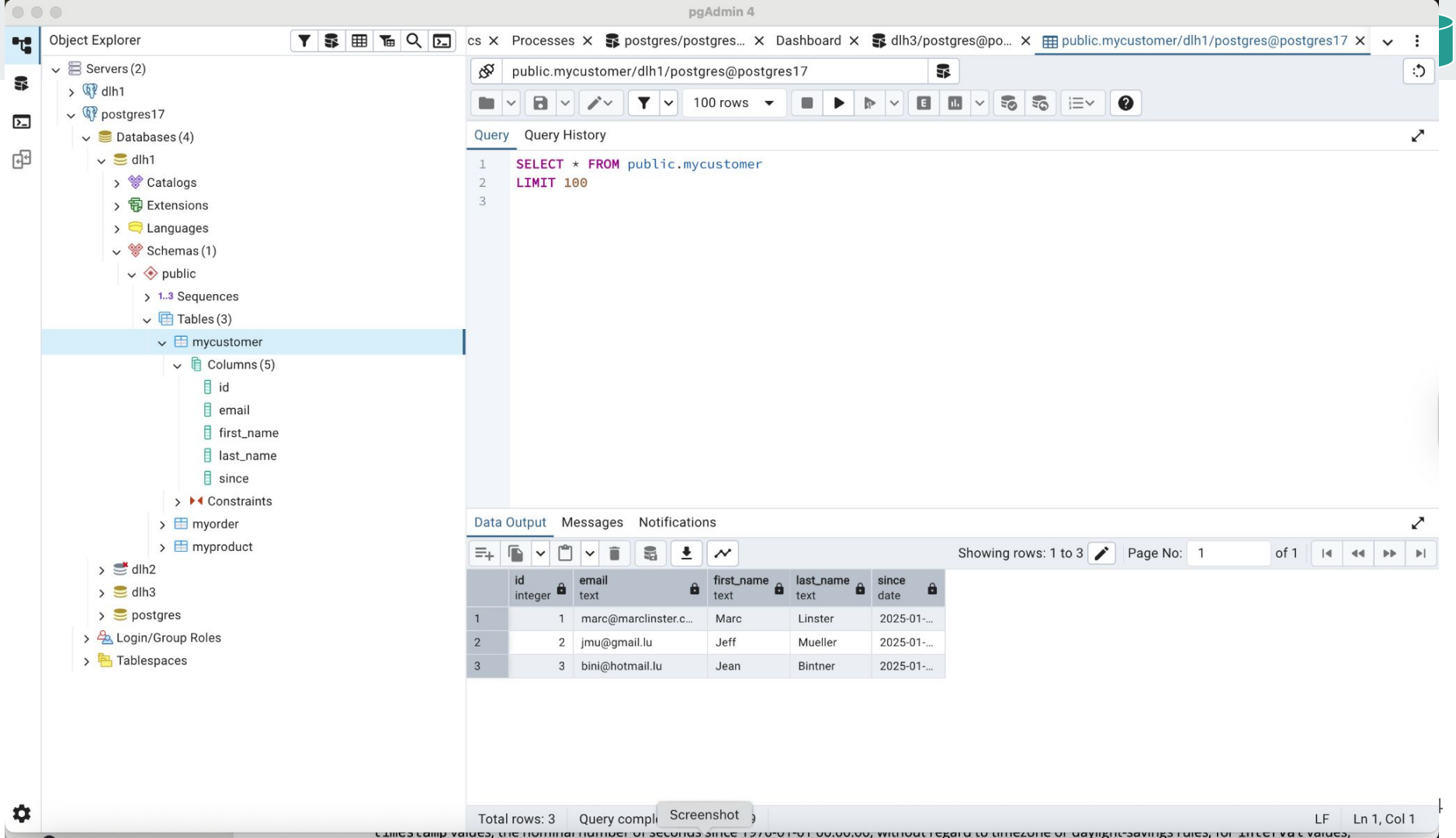
pgAdmin

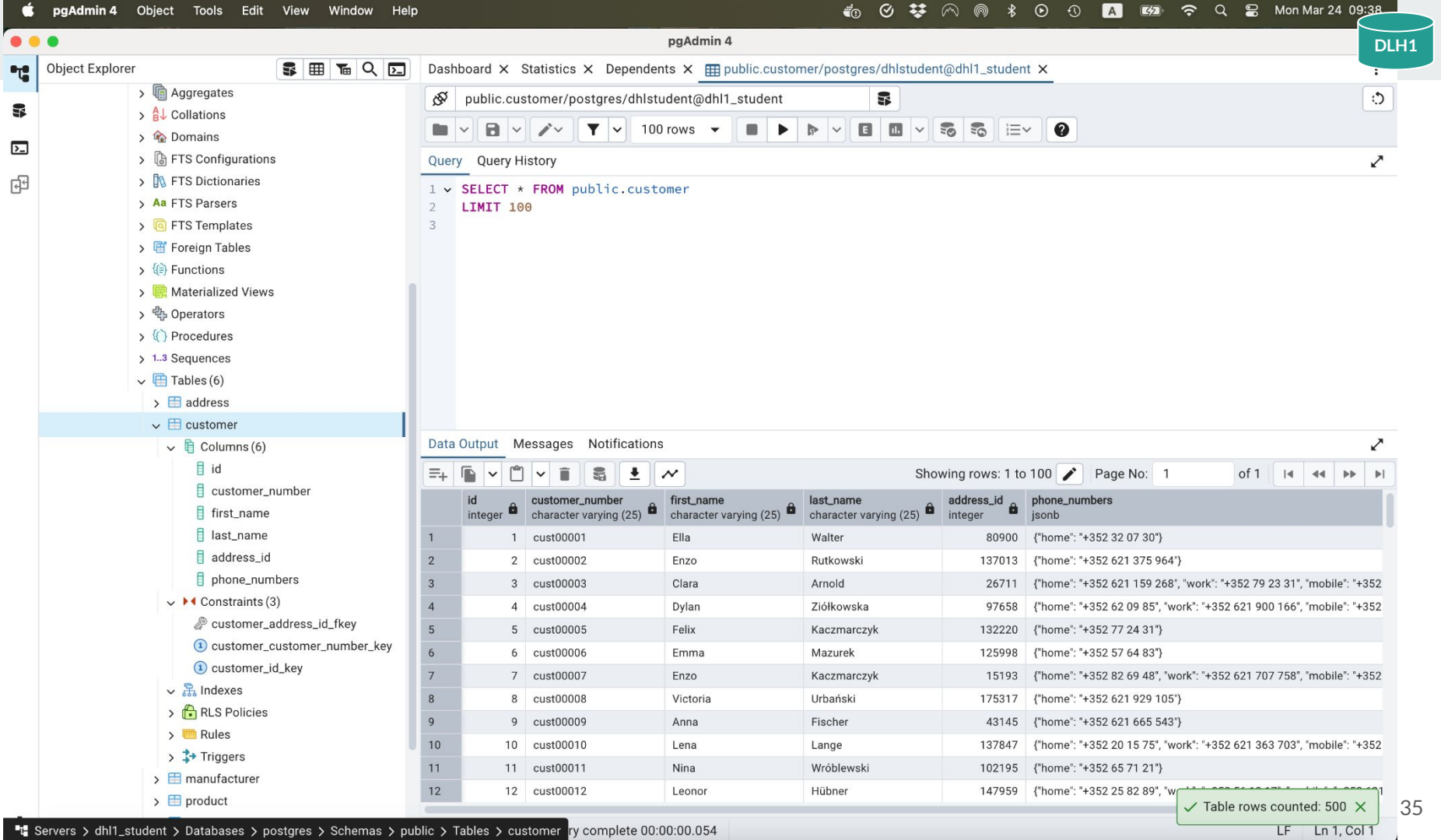

pgAdmin 4 Object Tools Edit View Window Help
Mon Mar 24 09:26
pgAdmin 4
Object Explorer
Servers (3)
Dashboard
Statistics
Dependents
Welcome
pgAdmin
Management Tools for PostgreSQL
Feature rich | Maximises PostgreSQL | Open Source
pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.
Quick Links
Add New Server
Configure pgAdmin
Getting Started
PostgreSQL Documentation
pgAdmin Website
Planet PostgreSQL
Community Support
Screenshot


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Welcome
pgAdmin
Management Tools for PostgreSQL
Feature rich | Maximises PostgreSQL | Open Source
pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.
Quick Links
Add New Server
Configure pgAdmin
Getting Started
PostgreSQL Documentation
pgAdmin Website
Planet PostgreSQL
Community Support


pgAdmin 4
Object Explorer
Servers (2)
dlh1
postgres17
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Sequences
Tables (3)
mycustomer
myorder
myproduct
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Statistics
Processes
postgres/postgres...
Dashboard
dlh3/postgres@postgres17
dlh3/postgres@postgres17
No limit
Query
Query History
1
Data Output
Messages
Notifications
Total rows:
Screenshot
LF
Ln 1, Col 1


pgAdmin 4 Object Tools Edit View Window Help
Mon Mar 24 09:33
pgAdmin 4
DLH1
Object Explorer
Dashboard
Statistics
Dependents
Please connect to the selected server to view the dashboard.
Servers (2)
dhl1
dhl1_student
Databases (1)
postgres
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (6)
address
customer
manufacturer
product
sales_transaction
store


pgAdmin 4
Object Explorer
Servers (2)
dlh1
postgres17
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Sequences
Tables (3)
mycustomer
Columns (5)
id
email
first_name
last_name
since
Constraints
myorder
myproduct
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
cs
Processes
postgres/postgres...
Dashboard
dlh3/postgres@po...
public.mycustomer/dlh1/postgres@postgres17
public.mycustomer/dlh1/postgres@postgres17
100 rows
Query
Query History
1 SELECT * FROM public.mycustomer
2 LIMIT 100
3
Data Output
Messages
Notifications
Showing rows: 1 to 3
Page No: 1
of 1
id integer
email text
first_name text
last_name text
since date
1 1 marc@marclinster.c... Marc Linster 2025-01-...
2 2 jmu@gmail.lu Jeff Mueller 2025-01-...
3 3 bini@hotmail.lu Jean Bintner 2025-01-...
Total rows: 3
Query compl
Screenshot
LF
Ln 1, Col 1


pgAdmin 4 Object Tools Edit View Window Help
Mon Mar 24 09:38
pgAdmin 4
DLH1
Object Explorer
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (6)
address
customer
Columns (6)
id
customer_number
first_name
last_name
address_id
phone_numbers
Constraints (3)
customer_address_id_fkey
customer_customer_number_key
customer_id_key
Indexes
RLS Policies
Rules
Triggers
manufacturer
product
Dashboard
Statistics
Dependents
public.customer/postgres/dhlstudent@dhl1_student
public.customer/postgres/dhlstudent@dhl1_student
100 rows
Query
Query History
1 SELECT * FROM public.customer
2 LIMIT 100
3
Data Output
Messages
Notifications
Showing rows: 1 to 100
Page No: 1
of 1
id integer
customer_number character varying (25)
first_name character varying (25)
last_name character varying (25)
address_id integer
phone_numbers jsonb
1 1 cust00001 Ella Walter 80900 {"home": "+352 32 07 30"}
2 2 cust00002 Enzo Rutkowski 137013 {"home": "+352 621 375 964"}
3 3 cust00003 Clara Arnold 26711 {"home": "+352 621 159 268", "work": "+352 79 23 31", "mobile": "+352
4 4 cust00004 Dylan Ziółkowska 97658 {"home": "+352 62 09 85", "work": "+352 621 900 166", "mobile": "+352
5 5 cust00005 Felix Kaczmarczyk 132220 {"home": "+352 77 24 31"}
6 6 cust00006 Emma Mazurek 125998 {"home": "+352 57 64 83"}
7 7 cust00007 Enzo Kaczmarczyk 15193 {"home": "+352 82 69 48", "work": "+352 621 707 758", "mobile": "+352
8 8 cust00008 Victoria Urbański 175317 {"home": "+352 621 929 105"}
9 9 cust00009 Anna Fischer 43145 {"home": "+352 621 665 543"}
10 10 cust00010 Lena Lange 137847 {"home": "+352 20 15 75", "work": "+352 621 363 703", "mobile": "+352
11 11 cust00011 Nina Wróblewski 102195 {"home": "+352 65 71 21"}
12 12 cust00012 Leonor Hübner 147959
Table rows counted: 500
Servers > dhl1_student > Databases > postgres > Schemas > public > Tables > customer
ry complete 00:00:00.054
LF
Ln 1, Col 1

Tables and Queries: Basics

Table Basics

- Rows = records (a.k.a. tuples in Postgres)
- Columns = data elements
- Example: mycustomer table
  - ID = a way to uniquely identify a customer ⇒ usually a large number
  - Email = the email address, a text field, sometimes limited in length
  - First name ⇒ a text field, sometimes limited in length
  - Last name ⇒ a text field, sometimes limited in length
  - Since => a date field

```
postgres=# SELECT * FROM mycustomer;
 id |        email         | first_name | last_name |   since
----+----------------------+------------+-----------+------------
  1 | marc@marclinster.com | Marc       | Linster   | 2025-01-01
  2 | jmu@gmail.lu         | Jeff       | Mueller   | 2025-01-02
  3 | bini@hotmail.lu      | Jean       | Bintner   | 2025-01-03
```

Tables

- All rows/records have the same columns,
  - E.g., for every customer we have id, first_name, last_name, since
- Every column has a data type, which constrains what data can be in that position
  - E.g., ‘since’ is always a date, first_name is always a text
- Most important data types
  - Boolean: True/False
  - Character types: char, varchar, text
  - Numeric: integer, floating point, …
  - Temporal types: date, time, timestamp, interval
  - UUID: Unique identifies
  - Array: arrays of strings, numbers, etc
  - JSON: structured documents
  - Special types for network addresses, XML, geographic points, etc
- Every table should have a primary key

https://www.postgresql.org/docs/current/datatype.html

Illustrating Select Postgres Data Types

Boolean: 'true' or 'false'

Character Types

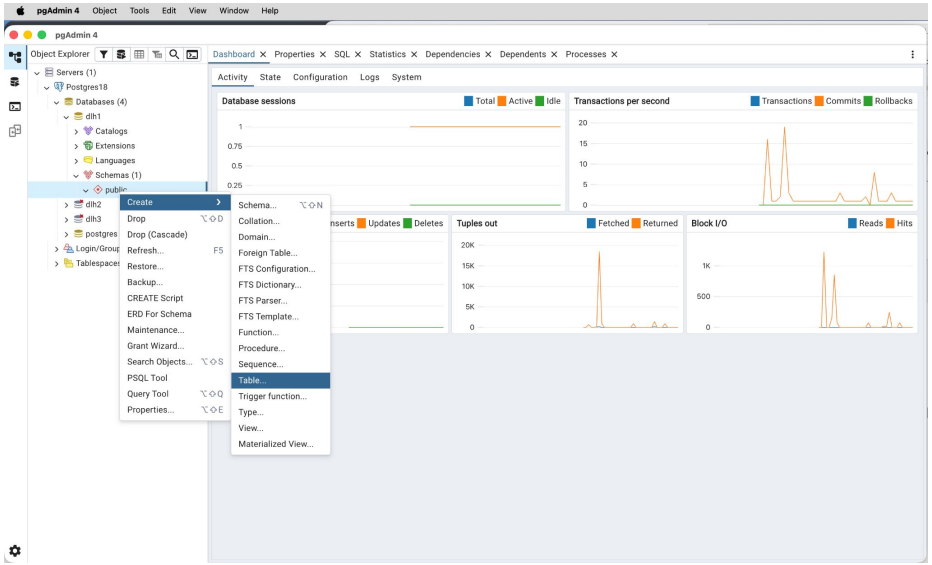
- Fixed length: CHARACTER(n), CHAR(n)
- Variable length w. length limit: CHARACTER VARYING (n), VARCHAR (n)
- Variable w.o. length limit: TEXT, VARCHAR

Number Types

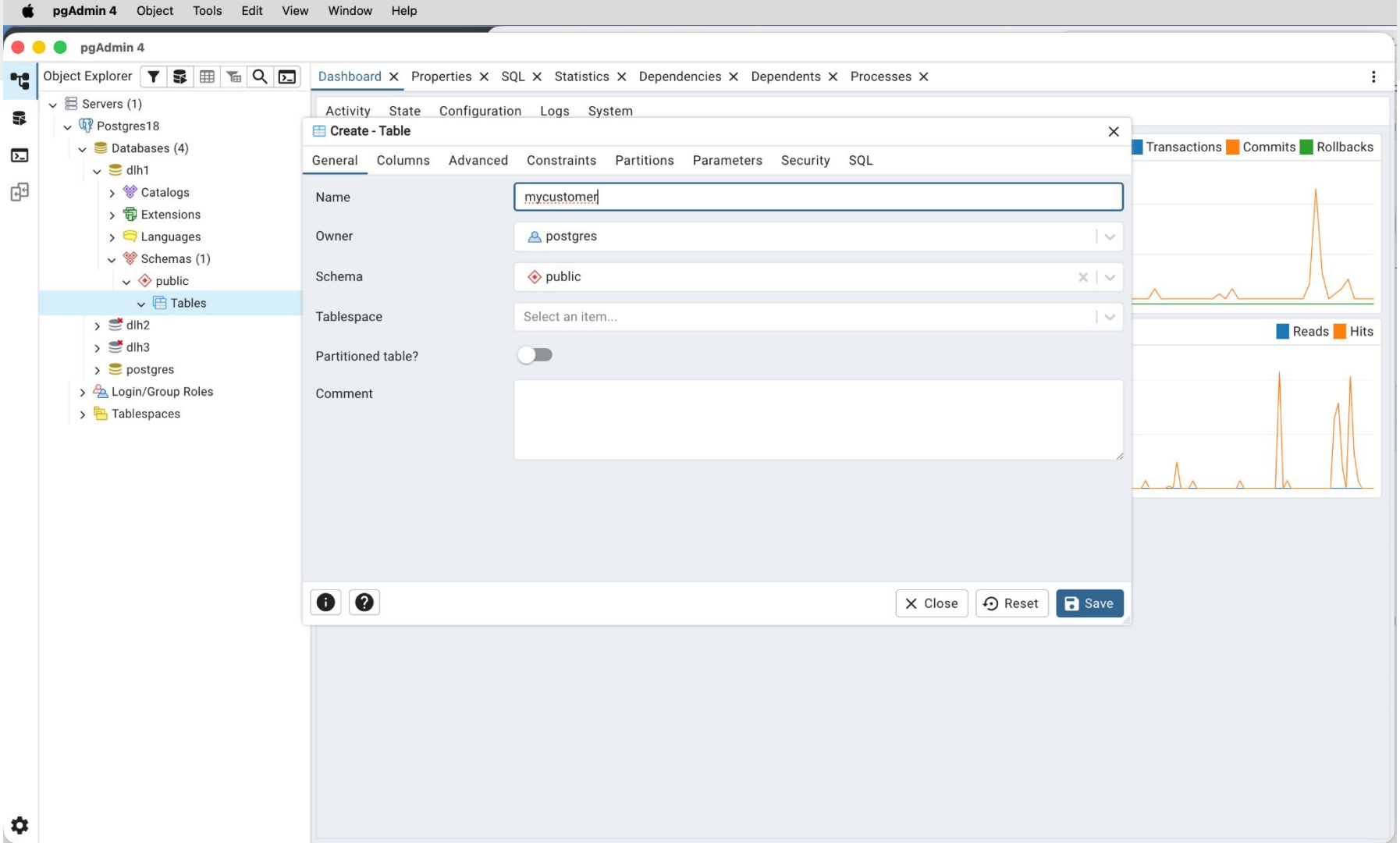
- INTEGER:
  - SMALLINT (+/- 32,768), INTEGER (+/- 2,147,483,648), BIGINT (+/- 9,223,372,036,854,775,808)
- NUMERIC/DECIMAL:
  - NUMERIC (5,2) - 5 digits before the period, two after the period, for example 172.99

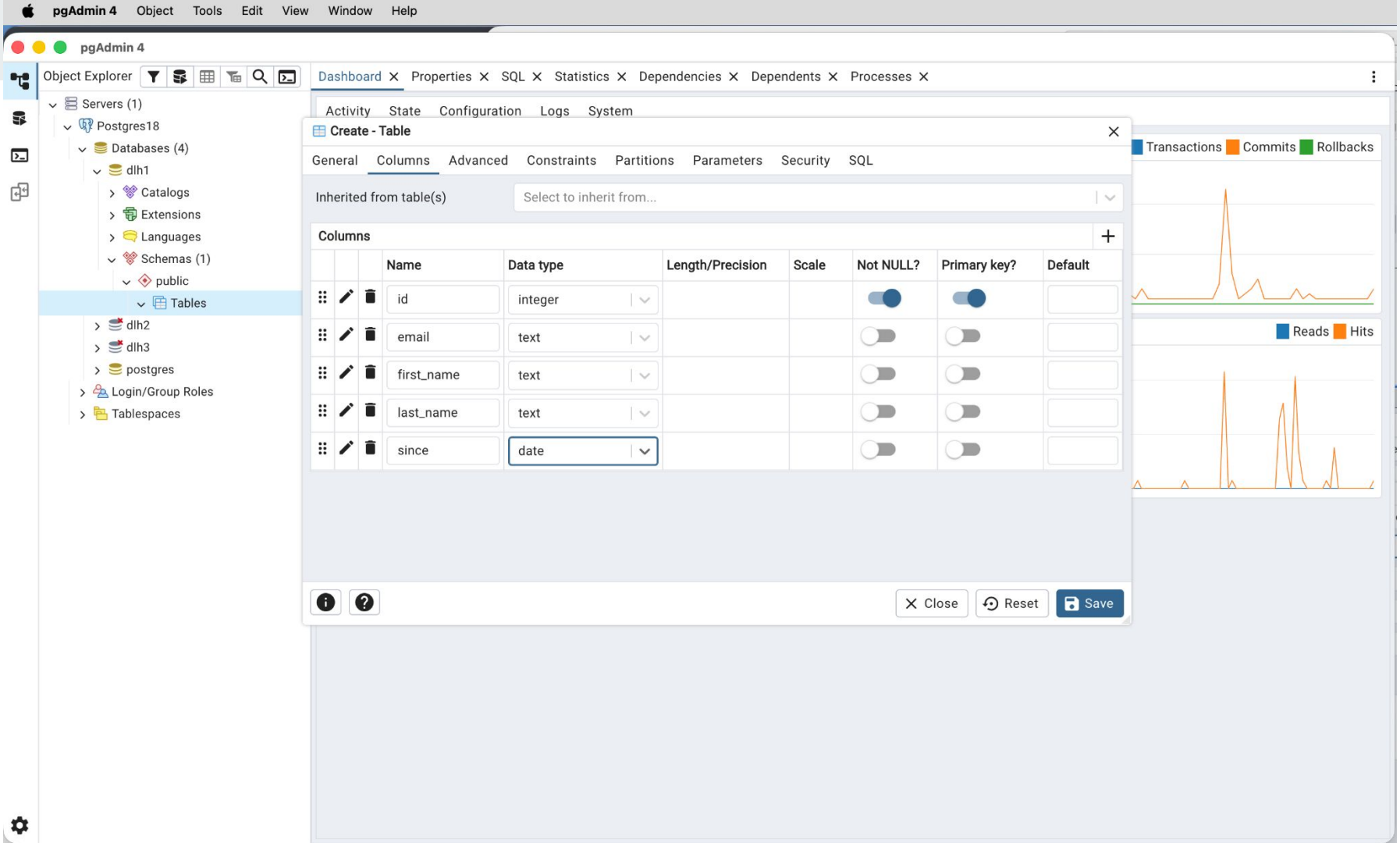
Let's Try it Out

- pgAdmin Query Tool
  - Make sure you are connected to dlh1!
    - Enter "`SELECT current_database()` to check where you connected
- We will create the table 'customer' with
  - id - integer - primary key
  - email - text
  - last_name - text
  - first_name - text
  - since - date



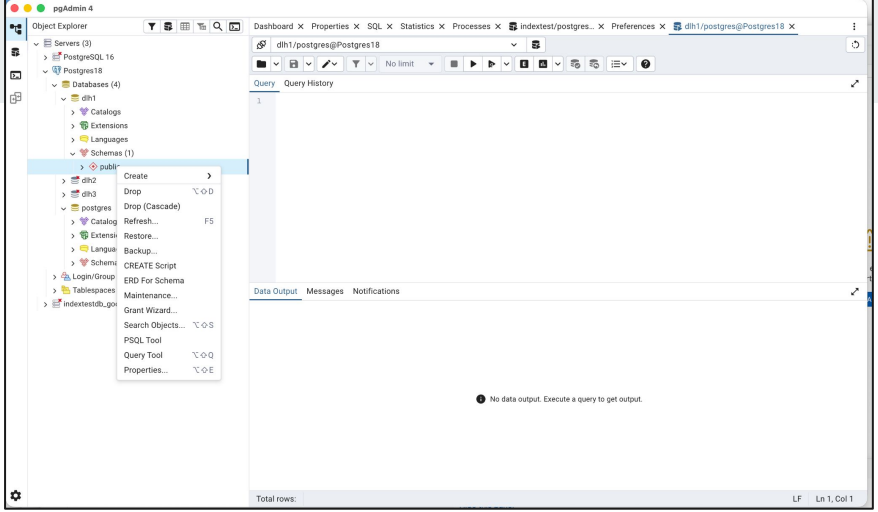
All names (tables and columns) are
- lower case
- no special characters (except '_')
- begin with a character


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Activity
State
Configuration
Logs
System
Create - Table
General
Columns
Advanced
Constraints
Partitions
Parameters
Security
SQL
Name
mycustomer
Owner
postgres
Schema
public
Tablespace
Select an item...
Partitioned table?
Comment
Close
Reset
Save
Transactions
Commits
Rollbacks
Reads
Hits


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Activity
State
Configuration
Logs
System
Create - Table
General
Columns
Advanced
Constraints
Partitions
Parameters
Security
SQL
Inherited from table(s)
Select to inherit from...
Columns
Name
Data type
Length/Precision
Scale
Not NULL?
Primary key?
Default
id
integer
email
text
first_name
text
last_name
text
since
date
Close
Reset
Save
Transactions
Commits
Rollbacks
Reads
Hits

Browse

- Check out the list of tables in the pgAdmin tree browser (hit refresh)

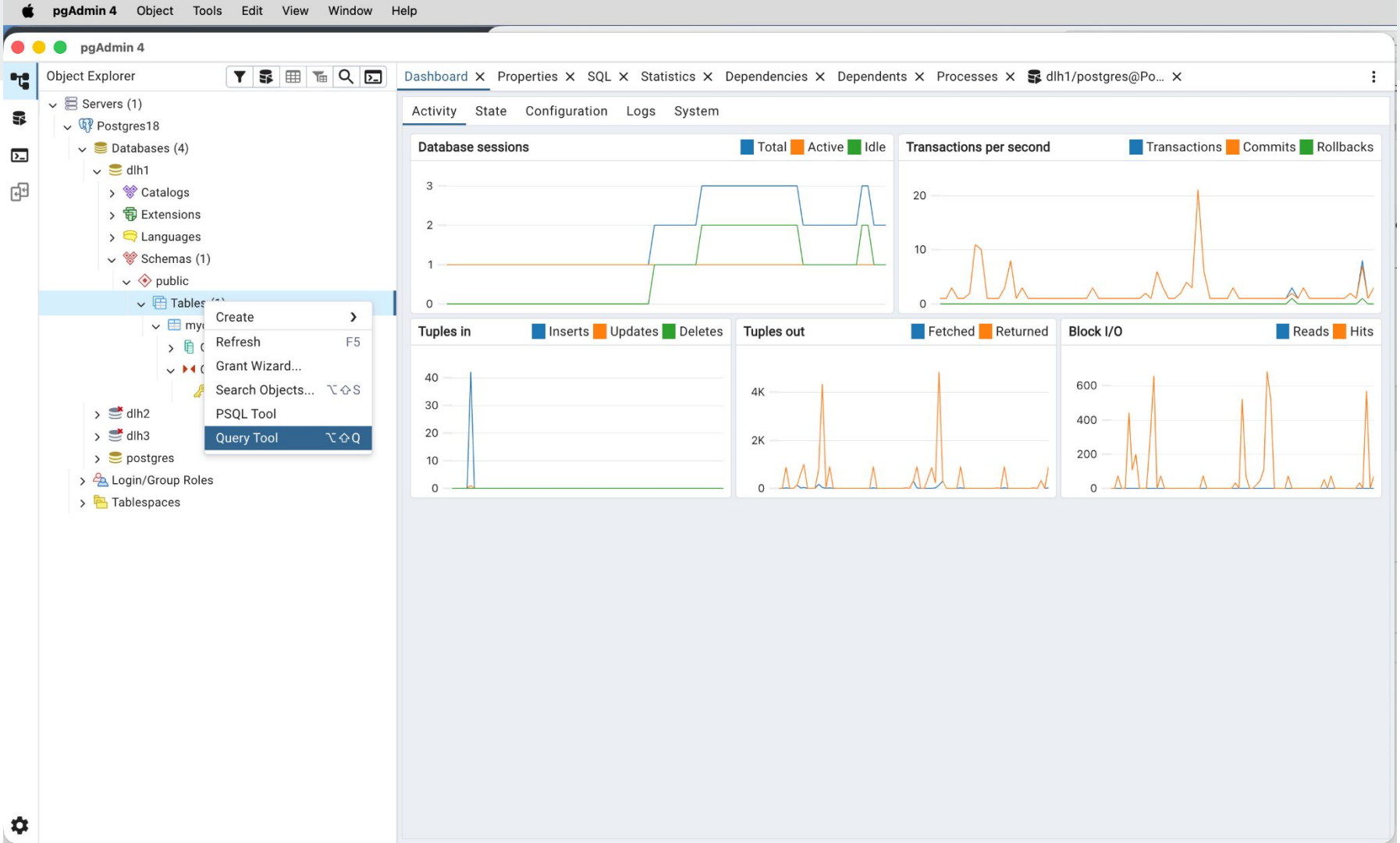


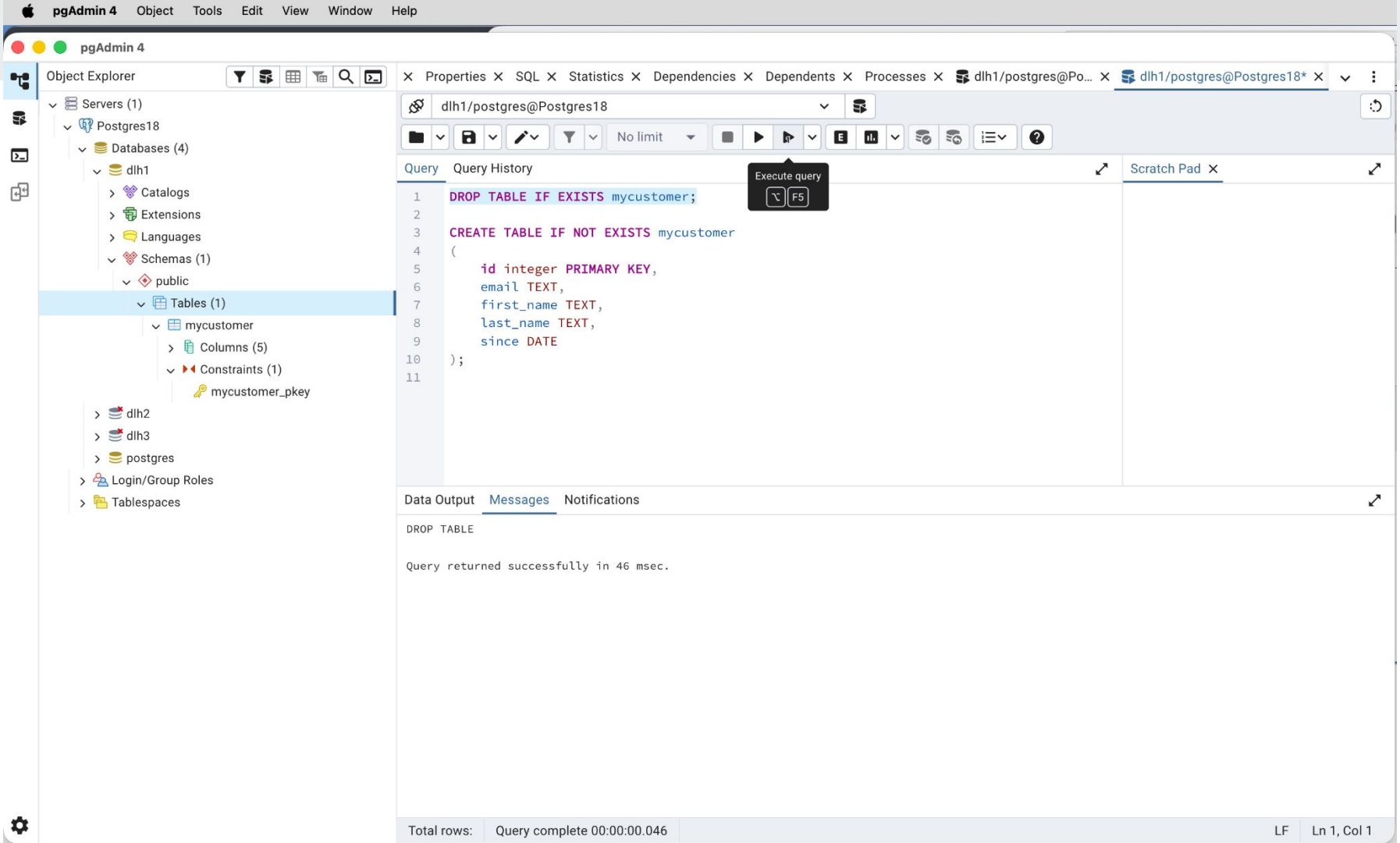
Lets try it with SQL

- Open a query tool
- Check if you are in the right database
- Run:

```
DROP TABLE IF EXISTS mycustomer;

CREATE TABLE IF NOT EXISTS mycustomer
(
    id integer PRIMARY KEY,
    email TEXT,
    first_name TEXT,
    last_name TEXT,
    since DATE
);
```


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables
Create
Refresh F5
Grant Wizard...
Search Objects... ⌥⇧S
PSQL Tool
Query Tool ⌥⇧Q
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
dlh1/postgres@Po...
Activity
State
Configuration
Logs
System
Database sessions
Total
Active
Idle
Transactions per second
Transactions
Commits
Rollbacks
Tuples in
Inserts
Updates
Deletes
Tuples out
Fetched
Returned
Block I/O
Reads
Hits


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables (1)
mycustomer
Columns (5)
Constraints (1)
mycustomer_pkey
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Properties
SQL
Statistics
Dependencies
Dependents
Processes
dlh1/postgres@Po...
dlh1/postgres@Postgres18*
dlh1/postgres@Postgres18
No limit
Query
Query History
Execute query
⌥ F5
Scratch Pad
1 DROP TABLE IF EXISTS mycustomer;
2
3 CREATE TABLE IF NOT EXISTS mycustomer
4 (
5 id integer PRIMARY KEY,
6 email TEXT,
7 first_name TEXT,
8 last_name TEXT,
9 since DATE
10);
11
Data Output
Messages
Notifications
DROP TABLE
Query returned successfully in 46 msec.
Total rows:
Query complete 00:00:00.046
LF
Ln 1, Col 1

Uppercase, lowercase, white spaces, etc

- Postgres ignores case - all names for tables, columns, functions, procedures etc. are lowercase
  - Postgres automatically lower cases all identifiers!!!
  - SQL identifiers and keywords must begin with a letter or an underscore (_).
  - Subsequent characters can be letters, underscores, digits (0-9), or dollar signs ($).
  - Note that dollar signs are not allowed in identifiers according to the letter of the SQL standard, so their use might render applications less portable.
- Customername = CustomerName = CUSTOMERNAME = customername <> "CustomerName" <> "customerName" <> "CUSTOMERNAME"
- Do not use camelCase, use snake_case to improve readability of identifiers, e.g. customer_name
- Spaces (' ') cannot be used in identifiers
- Double quotes "" can be used to create mixed case identifiers that include spaces

DON'T DO IT!!!
Mixed case and spaces cause all kinds of problems later on

Query Basics

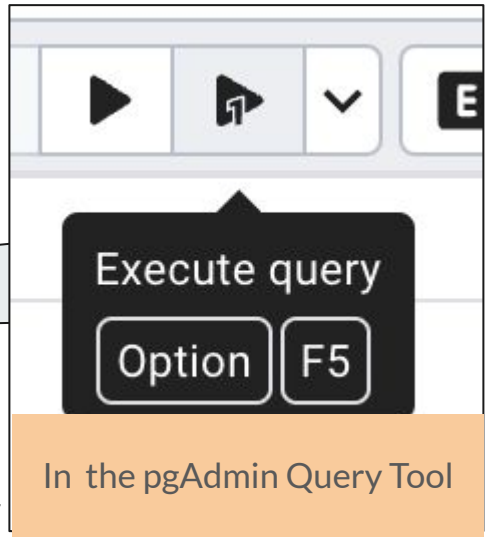
Query types

- SELECT: List data from tables that meet certain conditions
- UPDATE: Change data in records that meet certain conditions
- INSERT: Add records
- DELETE: Remove Records

Let's try it out

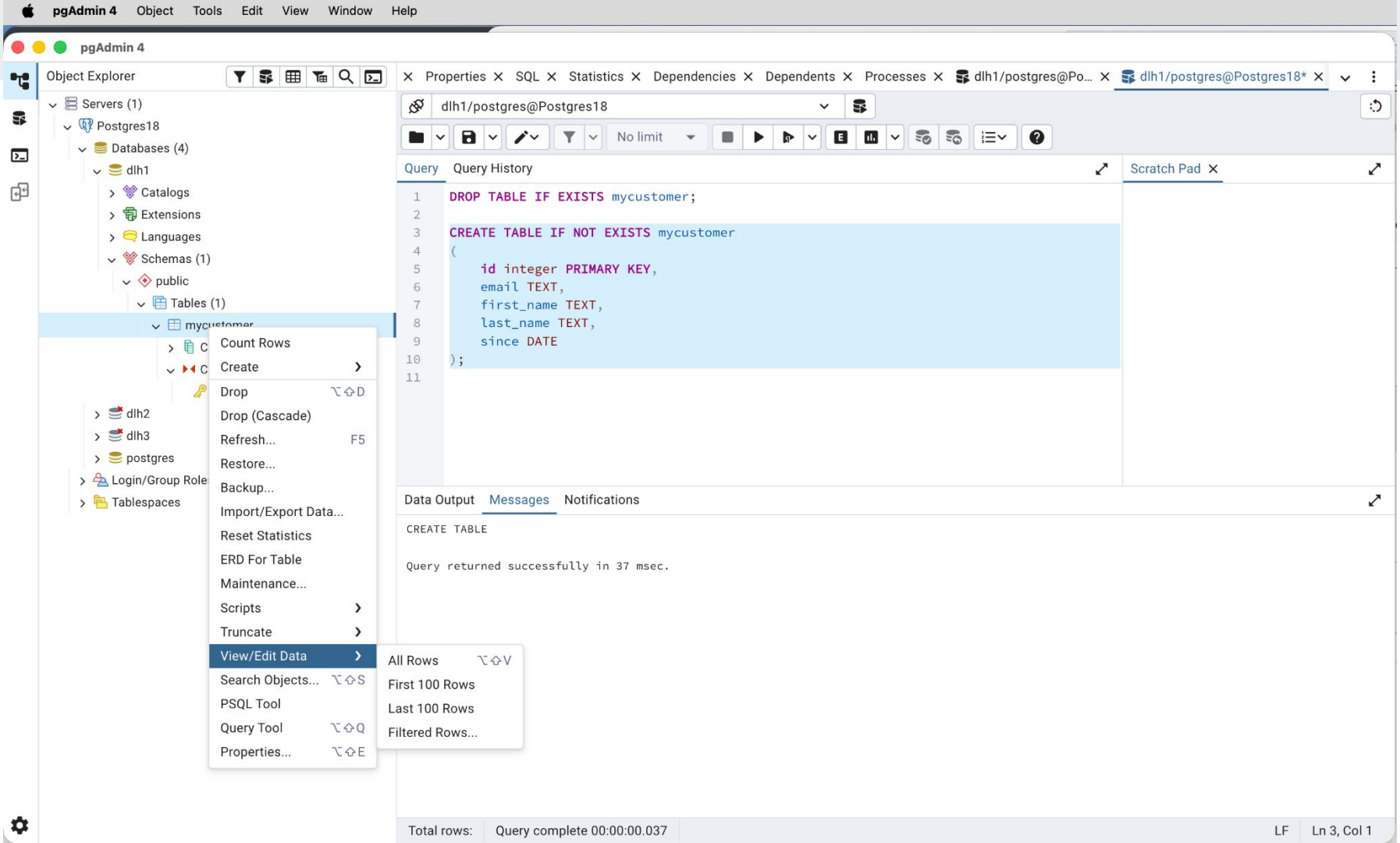
```
SELECT * FROM mycustomer;
```

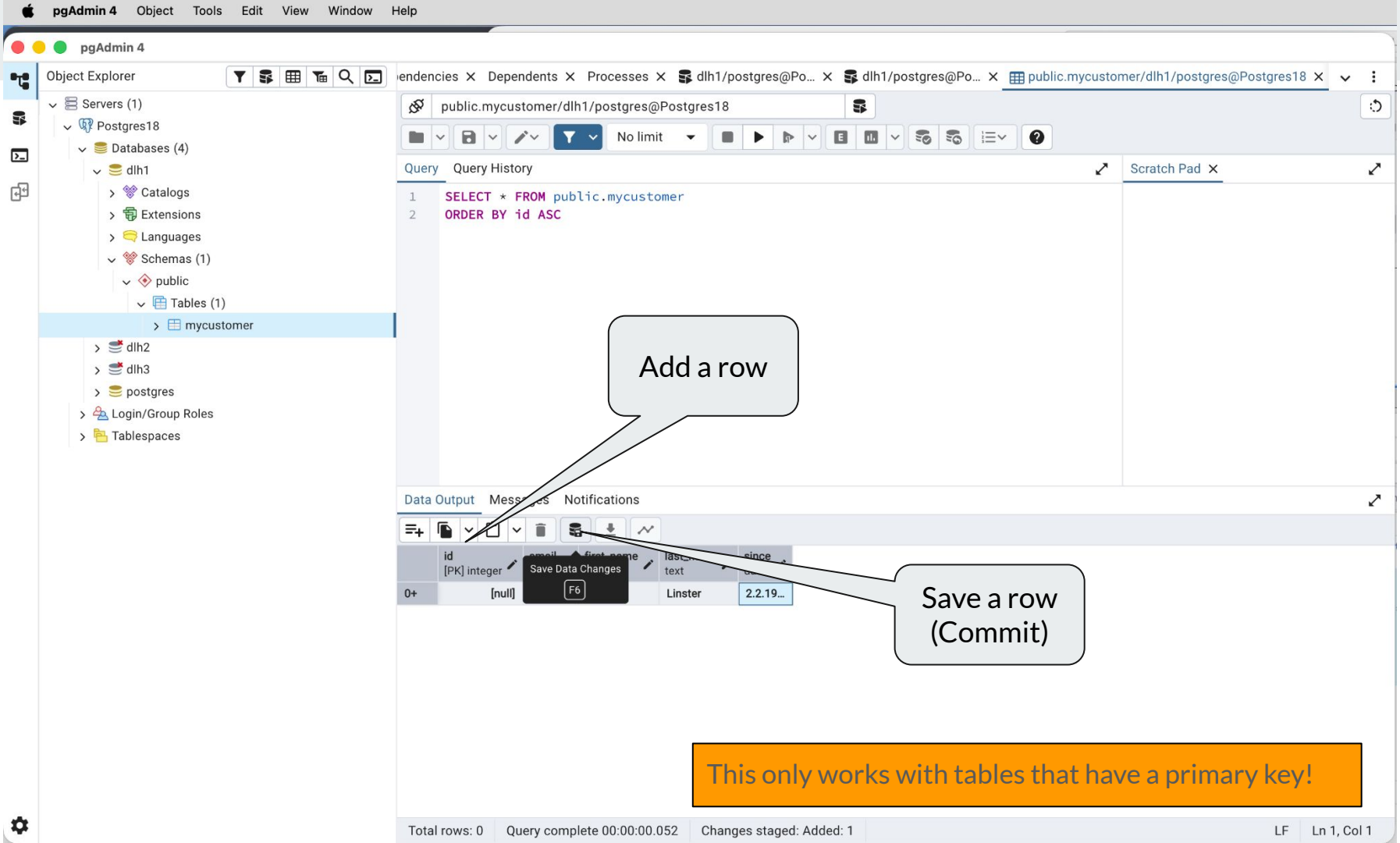
- Remember the ';'
- Click




```
INSERT INTO mycustomer (id, email, first_name, last_name, since)
     VALUES
           (1,'marc@marclinster.com', 'Marc', 'Linster','2025-01-01'),
           (2,'jmu@gmail.lu','Jeff', 'Mueller','2024-05-06'),
           (3,'bini@hotmail.lu', 'Jean','Bintner','2023-09-07');

SELECT first_name FROM mycustomer;
SELECT first_name, last_name FROM mycustomer;
SELECT * FROM mycustomer WHERE last_name = 'Linster';
SELECT * FROM mycustomer WHERE first_name LIKE 'Je%';
```


pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables (1)
mycustomer
dlh2
dlh3
postgres
Login/Group Role
Tablespaces
Count Rows
Create
Drop
Drop (Cascade)
Refresh...
Restore...
Backup...
Import/Export Data...
Reset Statistics
ERD For Table
Maintenance...
Scripts
Truncate
View/Edit Data
Search Objects...
PSQL Tool
Query Tool
Properties...
All Rows
First 100 Rows
Last 100 Rows
Filtered Rows...
Properties
SQL
Statistics
Dependencies
Dependents
Processes
dlh1/postgres@Postgres18
No limit
Query
Query History
Scratch Pad
DROP TABLE IF EXISTS mycustomer;
CREATE TABLE IF NOT EXISTS mycustomer
(
id integer PRIMARY KEY,
email TEXT,
first_name TEXT,
last_name TEXT,
since DATE
);
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 37 msec.
Total rows:
Query complete 00:00:00.037
LF
Ln 3, Col 1


pgAdmin 4 Object Tools Edit View Window Help
pgAdmin 4
Object Explorer
Servers (1)
Postgres18
Databases (4)
dlh1
Catalogs
Extensions
Languages
Schemas (1)
public
Tables (1)
mycustomer
dlh2
dlh3
postgres
Login/Group Roles
Tablespaces
Dependencies
Dependents
Processes
dlh1/postgres@Po...
dlh1/postgres@Po...
public.mycustomer/dlh1/postgres@Postgres18
public.mycustomer/dlh1/postgres@Postgres18
No limit
Query
Query History
Scratch Pad
1 SELECT * FROM public.mycustomer
2 ORDER BY id ASC
Add a row
Data Output
Messages
Notifications
id
[PK] integer
text
since
Save Data Changes
F6
0+
[null]
Linster
2.2.19...
Save a row
(Commit)
This only works with tables that have a primary key!
Total rows: 0
Query complete 00:00:00.052
Changes staged: Added: 1
LF
Ln 1, Col 1

Building Blocks of the simple SELECT Query

- SELECT
  - Columns to be selected, maybe using AS to provide a meaningful name
  - For example `last_name AS ln`
- FROM
  - Table(s) from which to select. Can also use AS to provide a new name
  - For example `customer AS c`
- WHERE
  - Filter clauses to select a subset of records
  - Can use the new names from the AS clause
  - For example `c.ln LIKE 'Li%'`
- ORDER BY
  - Organizes the output of the SELECT clause
  - Can use the new names from the AS clause
  - For example: `c.ln ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later

Let's try it out (use the pgAdmin Query Tool)

```
UPDATE mycustomer SET first_name = 'Mark' WHERE id = 1;
SELECT * FROM mycustomer;
DELETE FROM mycustomer WHERE id = 1;
INSERT INTO mycustomer (id, email, first_name, last_name, since)
   VALUES
       (1,'marc@marclinster.com', 'Marc', 'Linster','2025-01-01');
SELECT * FROM mycustomer;
SELECT * FROM mycustomer ORDER BY last_name ASC;
```

Day 2

- Review of Day 1
- Operators in the WHERE clause
- Calculations in the SELECT clause
- Primary Keys, Constraints, and Relationships

Relational Databases

- Tables, e.g., products, customers
- Relations, e.g., which customer bought which product
- Schemas, e.g., structured collections of tables and relations
- Data types, e.g., integer, numeric, float, text, time, date, …
- Data Definition Language (DDL), e.g. CREATE TABLE, CREATE INDEX, ….
- Data Manipulation Language (DML), e.g., SELECT name FROM product, INSERT (first_name, last_name) VALUES (‘marc’, ‘linster’) INTO customer
- Data Control Language (DCL), e.g., GRANT SELECT ON product
- Transaction Control Language (TCL), e.g., BEGIN TRANSACTION, COMMIT, ROLLBACK

Relational Databases

id	timestamp	product_id	customer_id	store_id	qty	unit_price	extended_price
80	2025-12-26 6:41:08	1405	279	1	61	208.23	12702.03
8	2025-12-07 7:20:22	392	75	4	71	203.4	14441.4
61	2025-12-08 22:50:43	1476	237	2	27	155.91	4209.57
59	2025-12-17 11:45:05	2232	331	3	6	132.73	796.38
10	2025-12-11 19:56:24	113	33	4	23	153.06	3520.38
66	2025-12-22 9:03:43	2178	70	1	61	125.83	7675.63
51	2025-12-07 21:30:30	2082	485	2	61	138.98	8477.78
52	2025-12-03 16:47:22	1105	443	3	69	49.7	3429.3
79	2025-12-19 21:46:07	236	126	4	15	81.04	1215.6
36	2025-12-27 10:37:25	1359	469	1	59	95.92	5659.28

name	description	manufacturer_id	size	color	sku	price
Cotton T-Shirt	Handcrafted details	3	XXL	Light Green	FSH-89166-000000000165	11.4
Wool Socks	Trendy and fashionable	1	M	Dark Yellow	FSH-53451-000000000138	146.26
Wool Dress Pants	Elegant and stylish	2	S	Turquoise	FSH-25089-0000000001213	102.79
Evening Gown	Waterproof and windproof	3	XS	Light Green	FSH-47771-000000000115	103.36
Wool Dress Pants	Elegant and stylish	2	XXL	Dark Blue	FSH-13141-000000000164	16.41
Baseball Cap	Versatile and functional	3	XL	Light Red	FSH-43094-000000000151	160.5
Slacks	Trendy and fashionable	3	XL	Dark Blue	FSH-82072-000000000154	39.63
Leather Jacket	Comfortable and durable	3	XS	Amber	FSH-15582-0000000001114	172.11
Sunglasses	Quick-drying	2	L	Dark Blue	FSH-65791-000000000144	18.73
Leather Belt	Comfortable and durable	3	XXL	Amber	FSH-11391-0000000001614	62.81

Uppercase, lowercase, white spaces, etc

- Postgres ignores case - all names for tables, columns, functions, procedures etc. are lowercase
  - Postgres automatically lower cases all identifiers!!!
  - SQL identifiers and keywords must begin with a letter or an underscore (_).
  - Subsequent characters can be letters, underscores, digits (0-9), or dollar signs ($).
  - Note that dollar signs are not allowed in identifiers according to the letter of the SQL standard, so their use might render applications less portable.
- Customername = CustomerName = CUSTOMERNAME = customername <> "CustomerName" <> "customerName" <> "CUSTOMERNAME"
- Do not use camelCase, use snake_case to improve readability of identifiers, e.g. customer_name
- Spaces (' ') cannot be used in identifiers
- Double quotes "" can be used to create mixed case identifiers that include spaces

DON'T DO IT!!!
Mixed case and spaces cause all kinds of problems later on

Building Blocks of the simple SELECT Query

- SELECT
  - Columns to be selected, maybe using AS to provide a meaningful name
  - For example `last_name AS ln`
- FROM
  - Table(s) from which to select. Can also use AS to provide a new name
  - For example `customer AS c`
- WHERE
  - Filter clauses to select a subset of records
  - Can use the new names from the AS clause
  - For example `c.ln LIKE 'Li%'`
- ORDER BY
  - Organizes the output of the SELECT clause
  - Can use the new names from the AS clause
  - For example: `c.ln ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later

Let's try it out (use the pgAdmin Query Tool)

```
UPDATE mycustomer SET first_name = 'Mark' WHERE id = 1;
SELECT * FROM mycustomer;
DELETE FROM mycustomer WHERE id = 1;
INSERT INTO mycustomer (id, email, first_name, last_name, since)
   VALUES
       (1,'marc@marclinster.com', 'Marc', 'Linster','2025-01-01');
SELECT * FROM mycustomer;
SELECT * FROM mycustomer ORDER BY last_name ASC;
```

Operators in the WHERE Clause

Postgres Operators in the WHERE Clause

`=`	Equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to
`<>`	Not equal to
`!=`	Not equal to
`LIKE`	Check if a value matches a pattern (case sensitive)
`ILIKE`	Check if a value matches a pattern (case insensitive)
`AND`	Logical AND
`OR`	Logical OR
`IN`	Check if a value is between a range of values
`BETWEEN`	Check if a value is between a range of values
`IS NULL`	Check if a value is NULL
`NOT`	Makes a negative result e.g. `NOT LIKE`, `NOT IN`, `NOT BETWEEN`

Let's try it out - Numbers

```
SELECT * FROM mycustomer;
SELECT * FROM mycustomer WHERE id = 1;
SELECT * FROM mycustomer WHERE id = 1 OR id = 3;
SELECT * FROM mycustomer WHERE id IN (1,2);
SELECT email, last_name, first_name
     FROM mycustomer
     WHERE id IN (1,2);
SELECT email, last_name, first_name
     FROM mycustomer
     WHERE id IN (1,2)
     ORDER BY last_name ASC;
```

Let's try it out - Text/Characters

```
SELECT * FROM mycustomer WHERE first_name = 'Jeff';

SELECT * FROM mycustomer WHERE first_name = 'jeff';

SELECT * FROM mycustomer WHERE first_name LIKE 'Je%';

SELECT * FROM mycustomer WHERE first_name LIKE 'je%';

SELECT * FROM mycustomer WHERE first_name ILIKE 'je%';

SELECT * FROM mycustomer WHERE first_name LIKE '_e%';
```

Let's try it out - Dates

```
SELECT last_name, first_name FROM mycustomer WHERE since = '2023-09-07';

SELECT last_name, first_name FROM mycustomer WHERE since > '2023-09-07';

SELECT last_name, first_name FROM mycustomer WHERE since >= '2023-09-07';

SELECT last_name, first_name, since
     FROM mycustomer
     WHERE since >= '2023-09-07'
     ORDER BY since DESC;
```

Computations in SELECT

Synonyms

- 'AS' defines a synonym
- Can be used for table names and column names
- Explicit form: `SELECT mc.email AS email_address FROM mycustomer AS mc;`
    - Using 'AS' makes the query more readable
    - Use snake_case for the pretty names
- Short form: `SELECT mc.email email_address FROM mycustomer mc;`

Working with strings in the SELECT clause

1) Appending strings

```
SELECT last_name || ', ' || first_name AS customer
     FROM mycustomer;
```

2) Formatting strings

```
SELECT
     FORMAT ('%s %s has been a valued customer since %s',
               first_name, last_name, since)
     FROM mycustomer;
```

3) Substrings

```
SELECT FORMAT ('%s. %s has been a valued customer since %s',
               SUBSTRING (first_name, 1,1), last_name, since)
     FROM mycustomer;
```

TO_CHAR – convert a timestamp or a numeric value to a string.

- SELECT TO_CHAR (NOW(), 'DAY');
- SELECT TO_CHAR (NOW(), 'MON');
- SELECT TO_CHAR (CURRENT_DATE, 'Day Month, DD, YYYY');
- SELECT TO_CHAR (12345.55, '99G999G999D99L');

Working with numbers in the SELECT clause

1) Simple arithmetic

```
SELECT 15 * 4;
SELECT 15 / 4;
SELECT 15 / 4.0;
SELECT MOD (15, 4);
```

2) Casting

```
SELECT PI();
SELECT PI()::INTEGER;
SELECT PI()::NUMERIC;
```

3) Rounding

```
SELECT ROUND(PI()::NUMERIC);
SELECT ROUND(PI()::NUMERIC, 2);
```

Working with dates in the SELECT clause

1) Extracting date elements

```
SELECT last_name, since, EXTRACT (DAY FROM since) FROM mycustomer;
SELECT last_name, since, EXTRACT (YEAR FROM since) FROM mycustomer;
SELECT last_name, since, EXTRACT (DOW FROM since) FROM mycustomer;
```

2) Differences

```
SELECT last_name, since,  NOW()::date - since FROM mycustomer;
SELECT last_name, since, AGE( NOW(), since) FROM mycustomer;
SELECT last_name, since,
      EXTRACT (YEAR FROM AGE(NOW(), since)) AS nbr_years
      FROM mycustomer;
```

EXTRACT

- EXTRACT from timestamp or date
  - CENTURY
  - DAY (day of the month)
  - DOW (day of the week - Sunday(0) to Saturday (6)
  - ISODOW (ISO standard day of the week - Monday (1) to Sunday (7)
  - DOY (day of the year)
  - HOUR, MINUTE, ….
- Try it out:
  - `SELECT 'It is the ' || EXTRACT (DOY FROM current_date) || 'th day of the year';`

```
             ?column?
--------------------------------
 It is the 234th day of the year
```

- Name the column using AS
  - `SELECT 'It is the ' || EXTRACT (DOY FROM current_date) || 'th day of the year' as day_of_year;`

Fancy stuff

```
SELECT FORMAT (
          '%s. %s is our oldest customer. '
          'He has been a valued customer for %s years, since %s',
          SUBSTRING (first_name, 1,1),
          last_name,
          EXTRACT (YEAR FROM AGE( since, NOW())),
          since)
     FROM mycustomer
     ORDER BY since ASC
     LIMIT 1;
```

Casting

- '::<data type>' allows to transform certain data types

```
SELECT NOW();

SELECT NOW()::DATE;
```

Try it out

- Create a query that
  - Selects customers from the customer table
  - Puts the name in the format ‘last name, first name’, e.g., Linster, Marc
  - Adds a columns entitled 'customer_since'
  - Sorts descending by last name

This code is not in the sample file - please write it on your own!

Primary Keys, Constraints and Relationships

Constraints and Relationships

- Constraints and relationships make sure your data in the tables is consistent
  - For example:
    - Orders always have quantities > 0
    - Products with prices >= 0
    - Products each having different names
    - Orders referring to existing products
- Constraints and relationships complement the definition of tables, columns, and data types

Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price >= 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
  - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
  - You can assign a ‘trigger’ to handle deletions

Primary Keys

- A unique way to find a row in the table
  - Cannot be NULL
  - Must be unique
- Can be integer, text, …
- Can be a combination of columns

id	email	first_name	last_name	since
1	marc@marclinster.com	Marc	Linster	2025-01-01
2	jmu@gmail.lu	Jeff	Mueller	2025-01-02
3	bini@hotmail.lu	Jean	Bintner	2025-01-03

This is what we will do to do

- 3 tables
  - mycustomer
  - myproduct
  - myorder
- Orders refer to customers and products
  - Every order links to one customer and one product (simplified model)
  - Make sure only existing customer can place an order
  - Make sure that all products actually exist

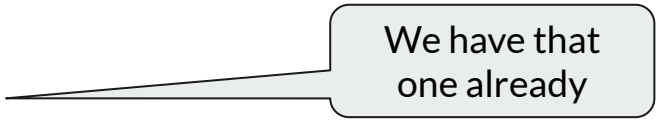


Table Basics

```
CREATE TABLE myproduct (
    product_nbr VARCHAR(10),
    name TEXT,
    price numeric
);
```

Problems:

- Every product should have a unique id (or product number)
- No two products should have the same name
- Every product should have a price > 0

Table Basics

```
DROP TABLE IF EXISTS myproduct;

CREATE TABLE myproduct (
     nbr VARCHAR(10) PRIMARY KEY,
     name TEXT UNIQUE NOT NULL,
     price NUMERIC NOT NULL CHECK (price > 0)
);
```

Try it out

- Insert 4 product records
- Use SELECT to make sure it worked
- Sort them by price ascending

Use the GUI or the Query Tool

- Add primary key to uniquely identify every record
  - PRIMARY KEY = UNIQUE + NOT NULL + indexed
  - Only one primary key per table
- Make sure that no products can have the same name
- All prices have to be greater than 0

```
SELECT * FROM myproduct;
 nbr          |           name           | price
--------------+--------------------------+-------
 ham_001      | Hammer 200gr             | 19.95
 ham_002      | Hammer 1kg               | 25.00
 chisel_004   | Chisel stone             | 39.95
 pli_019      | Pliers red, adjustable   | 13.50
```

CREATE TABLE myorder

- Introducing new capabilities:
  - Automatically generate a primary key
  - Link the order to a customer
  - Link the order to a product



Relations and foreign keys

```
DROP TABLE IF EXISTS myorder;

CREATE TABLE myorder (
      id INT PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
      date DATE,
      customer_id INTEGER NOT NULL,
      product_nbr VARCHAR(10) NOT NULL,
      qty INTEGER NOT NULL CHECK (qty > 0)
);
```

Nonsense order:

```
INSERT INTO myorder (date, customer_id, product_nbr, qty)
      VALUES (current_date, 25, 'big_thing', 11);
```

- Create PRIMARY KEY
- Automatically generate a unique ID

Problems:

- No guarantee that the product_nbr is actually in the `myproduct` table
- Not sure if the customer is actually defined!

Relations and foreign keys

```
DROP TABLE IF EXISTS myorder;

CREATE TABLE myorder (
      id INT
            PRIMARY KEY
            GENERATED BY DEFAULT AS IDENTITY,
      date DATE,
      customer_id INTEGER
            NOT NULL
            REFERENCES mycustomer (id),
      product_nbr VARCHAR(10)
            NOT NULL
            REFERENCES myproduct (nbr),
      qty INTEGER NOT NULL CHECK (qty > 0)
);
```

Is this clear?

try to create an order for a non-existing product

```
INSERT INTO myorder
      (date,customer_id, product_nbr, qty)
      VALUES ('2.2.2025', 1, 'cheese_10', 9);
;
```

```
ERROR:  insert or update on table "myorder"
violates foreign key constraint
"myorder product nbr fkey"
Key (product_nbr)=(cheese_10) is not present
in table "myproduct".

SQL state: 23503
Detail: Key (product_nbr)=(cheese_10) is not
present in table "myproduct".
```

```
SELECT * FROM myproduct;
    nbr     |          name          | price
------------+------------------------+-------
 ham_001    | Hammer 200gr           | 19.95
 ham_002    | Hammer 1kg             | 25.00
 chisel_004 | Chisel stone           | 39.95
 pli_019    | Pliers red, adjustable | 13.50
```

```
SELECT * FROM mycustomer;
 id |        email         | first_name | last_name |   since
----+----------------------+------------+-----------+------------
  1 | marc@marclinster.com | Marc       | Linster   | 2025-01-01
  2 | jmu@gmail.lu         | Jeff       | Mueller   | 2025-01-02
  3 | bini@hotmail.lu      | Jean       | Bintner   | 2025-01-03
```

Let's add orders

```
\d myorder
                                  Table "public.myorder"
   Column    |          Type          | Collation | Nullable |              Default
-------------+------------------------+-----------+----------+-----------------------
 id          | integer                |           | not null | generated by default …
 date        | date                   |           |          |
 customer_id | integer                |           |          |
 product_nbr | character varying(10)  |           |          |
 qty         | integer                |           | not null |
Indexes: "myorder_pkey" PRIMARY KEY, btree (id)
Check constraints: "myorder_qty_check" CHECK (qty > 0)
Foreign-key constraints:
    "myorder_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCE
    "myorder_product_nbr_fkey" FOREIGN KEY (product_nbr) REFEREN
```

Use the GUI or the Query Tool

```
INSERT INTO myorder (date, customer_id, product_nbr, qty)
     VALUES
          (NOW()::DATE, 1, 'ham_001', 1),
          (NOW()::DATE, 1, 'ham_002', 1),
          (NOW()::DATE-1, 2, 'chisel_004', 2),
          ('2025-03-01', 2, 'pli_019', 4);
```

Queries that work on multiple tables

SELECT <column_names> ⇐ comma separated list

FROM <table_names> ⇐ comma separated list

WHERE

ORDER BY

} Optional

```
SELECT * FROM myorder o,
          myproduct p,
          mycustomer c
     WHERE
          o.product_nbr = p.nbr
     AND
          o.customer_id = c.id;
```

Showing rows: 1 to 5 Page No: 1 of 1

	id integer	date date	customer_id integer	product_nbr character varying	qty integer	nbr character varyir	name text	price numeric	id integer	email text	first_name text	last_name text	since date
1	10	2025-02-02	1	chisel_004	9	chisel_004	Chisel stone	39.95	1	marc@marcli...	Marc	Linster	2025-01-...
2	11	2026-01-12	1	ham_001	1	ham_001	Hammer 200gr	19.95	1	marc@marcli...	Marc	Linster	2025-01-...
3	12	2026-01-12	1	ham_002	1	ham_002	Hammer 1kg	25.00	1	marc@marcli...	Marc	Linster	2025-01-...
4	13	2026-01-11	2	chisel_004	2	chisel_004	Chisel stone	39.95	2	jmu@gmail.lu	Jeff	Mueller	2024-05-...
5	14	2025-03-01	2	pli_019	4	pli_019	Pliers red, adjusta...	13.50	2	jmu@gmail.lu	Jeff	Mueller	2024-05-...

Try it out

1) Add products to myproduct table (up to 5 total)
2) Add several customers (max 5)
3) Add at least 5 orders for different products and customers. Orders dating in February 2025
4) Create a query that displays the orders with
    - Order id
    - Order date (ascending)
    - Customer name in format 'last_name, first_name'
    - Product name
    - Product price
    - Order quantity
    - Extended price (quantity * price)

Data Types in Postgres

Key Data Types in SQL

Boolean	Logical true or false	true, false
Numbers	Integers: Decimals Floating-point	1,2, 45, 987 (smallint, integers, bigint) 7.25 3.14159265
Binary	Raw data	PDF, images
Character types	Fixed/limited length Unlimited length (max 1 GB)	
Date time	Date Time Timestamp Interval	2025-09-23 16:49:47.70598 2025-09-23 16:49:47.70598 1 day
JSON/JSONB	JSON/JSONB	{"home": "47 25 07", "mobile": "621 599-314"}
Array	INTEGER[]	{1,2,5,19,27}
Ranges	Date ranges Number ranges	[2025-03-01, 2025-03-31) [1, 9]
UUID	Universally Unique IDs	01995196-02ef-7db9-b11b-b906ee7b4926

Numbers

Data Type	Range	Storage	Pro	Con
SMALINT	+/- 32,768	2 bytes	Fast computation Small storage	No fractions Automatic rounding
INTEGER	+/- 2,147 M	4 bytes		
BIGINT	+/- 9,223 B	8 bytes		
NUMERIC/ DECIMAL	Max. 131072 digits bef. decimal point; max 16383 digits aft. decimal point	Variable	No rounding errors	More storage Slower calculations
REAL	-3.40282347 × 1E38 to 3.40282347 × 1E38	4 bytes	Imprecise w. rounding errors	Fast calculations Less storage than NUMERIC/DECIMAL
DOUBLE PRECISION	1E-307 to 1E+308	8 bytes		

Casting

Get the complete list of all possible casts

```
SELECT
    format_type(c.castsource, NULL) AS source_type,
    format_type(c.casttarget, NULL) AS target_type
FROM pg_cast c
JOIN pg_type s ON c.castsource = s.oid
JOIN pg_type t ON c.casttarget = t.oid
ORDER BY 1, 2;
```

- CAST(value) uses the cast function
- '`value::target_type`' uses the cast operator
- Casting converts one data type to another (if allowed)
- Examples
  - `SELECT '100'::INTEGER;`
  - `SELECT '100x'::INTEGER;`
  - `SELECT NOW()::DATE;`
  - `SELECT '2026-11-30'::DATE;`
  - `SELECT '2026-11-31'::DATE;`
  - ```
    SELECT CAST('true' AS BOOLEAN),
           CAST('false' as BOOLEAN),
           CAST('T' as BOOLEAN),
           CAST('F' as BOOLEAN);
    ```
    ```

Day 3

- Review of Day 2
- The NULL problem
- Organizing data - schemas and entity relationship diagrams
- Data normalization and avoiding redundancy
- Queries revisited: Joins
- Views

Recap

Query types

- SELECT: List data from tables that meet certain conditions
- UPDATE: Change data in records that meet certain conditions
- INSERT: Add records
- DELETE: Remove Records

Building Blocks of the simple SELECT Query

- SELECT
  - Columns to be selected, maybe using AS to provide a meaningful name
  - For example `last_name AS ln`
- FROM
  - Table(s) from which to select. Can also use AS to provide a new name
  - For example `customer AS c`
- WHERE
  - Filter clauses to select a subset of records
  - Can use the new names from the AS clause
  - For example `c.ln LIKE 'Li%'`
- ORDER BY
  - Organizes the output of the SELECT clause
  - Can use the new names from the AS clause
  - For example: `c.ln ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later

Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price > 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
  - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
  - You can assign a ‘trigger’ to handle deletions

The 'NULL' problem

NULL <> ''

```
CREATE TABLE mytest (
   id INTEGER PRIMARY KEY,
   value1 INTEGER,
   value2 TEXT
);

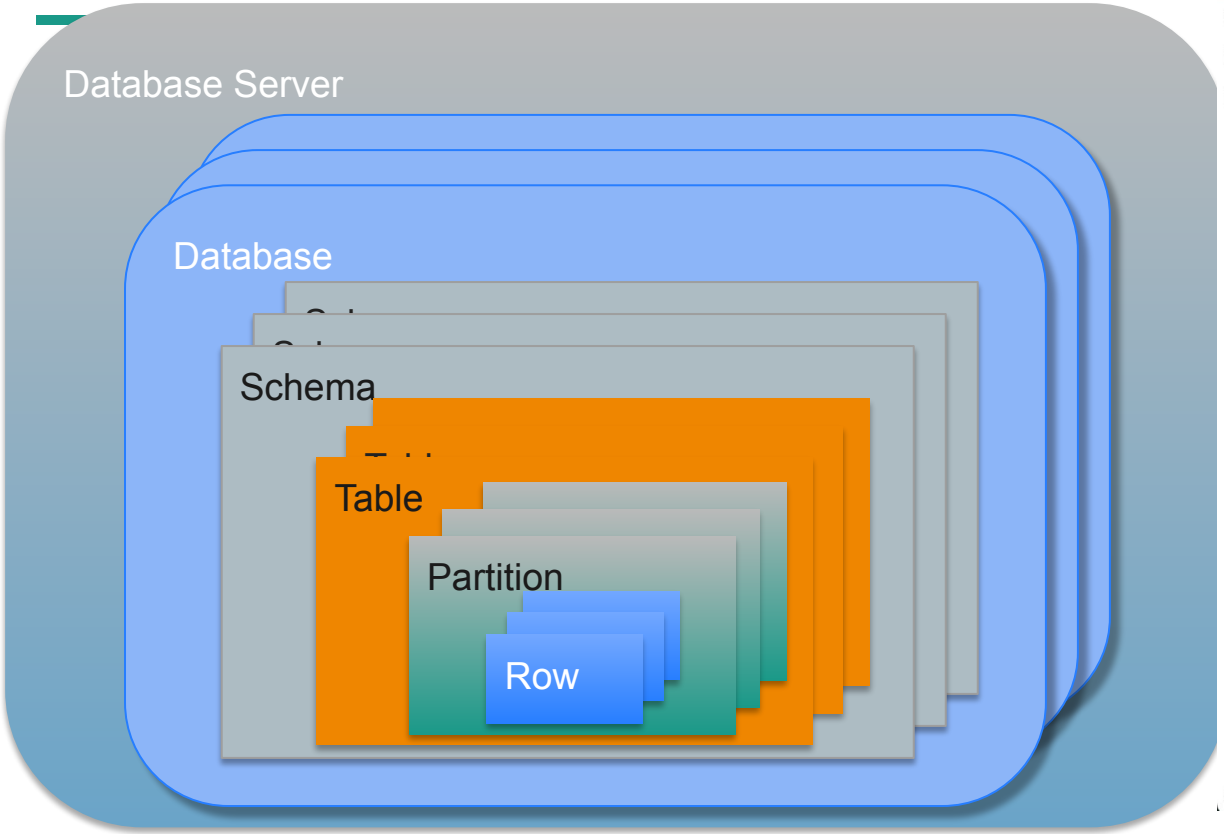
INSERT INTO mytest (id, value1, value2) VALUES (1, 10, '20');
INSERT INTO mytest (id, value1, value2) VALUES (2, NULL, '20');
INSERT INTO mytest (id, value1, value2) VALUES (3, 0, NULL);
INSERT INTO mytest (id, value1, value2) VALUES (4, 10, '');
INSERT INTO mytest (id, value1) VALUES (5, NULL);

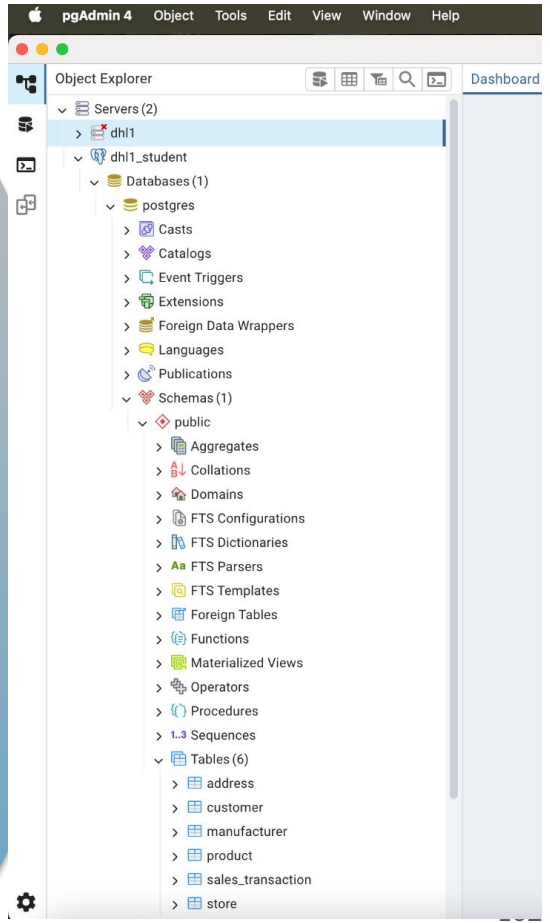
SELECT * FROM mytest;
SELECT * FROM mytest WHERE value1 < 15;
SELECT * FROM mytest WHERE value2 < '30';
SELECT * FROM mytest WHERE value2 IS NULL;
SELECT * FROM mytest WHERE value2 IS NOT NULL;
SELECT * FROM mytest WHERE value2 = '';

SELECT id, value1, value2 FROM mytest;
SELECT id, COALESCE(value1, -1) as value1, COALESCE(value2, 'this is NULL') as value2 FROM mytest;
```

Organizing Data - Schemas and Entity Relationship Diagrams

Hierarchy of Postgres and SQL

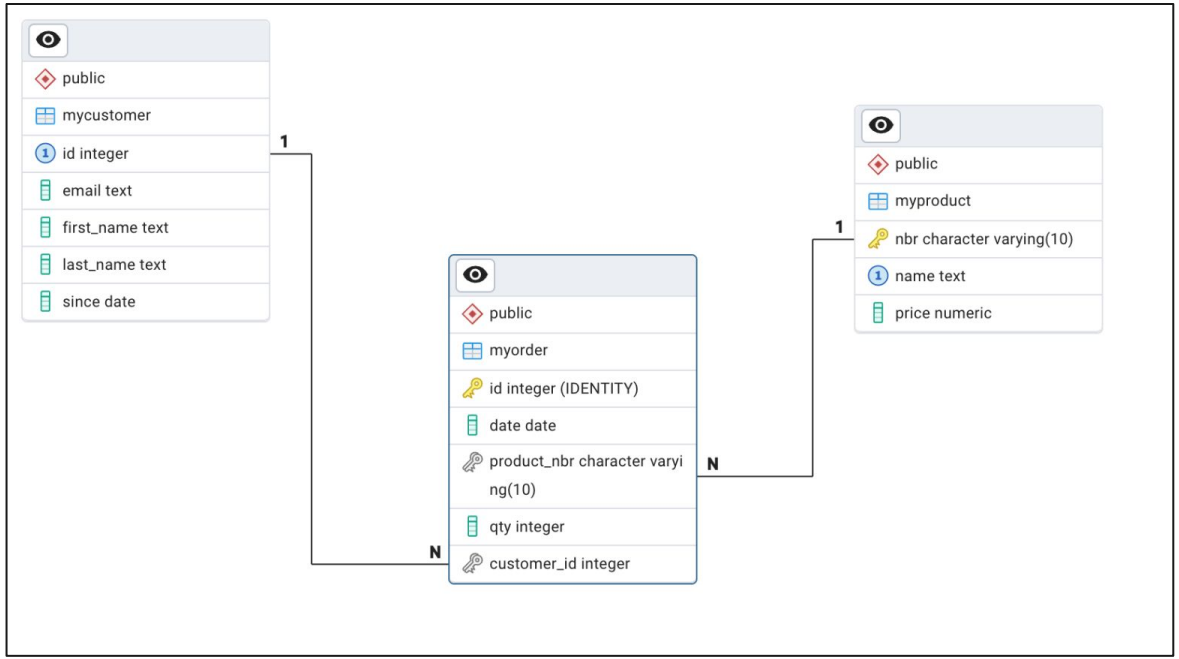




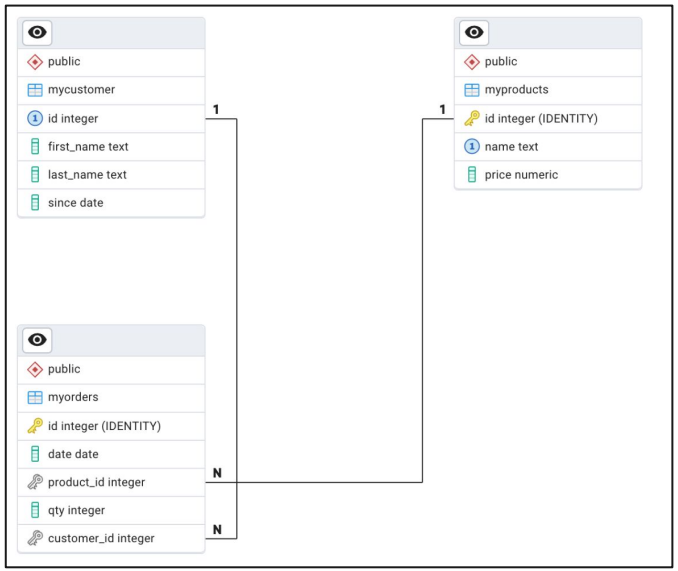
Schemas

Graphical view of the schema

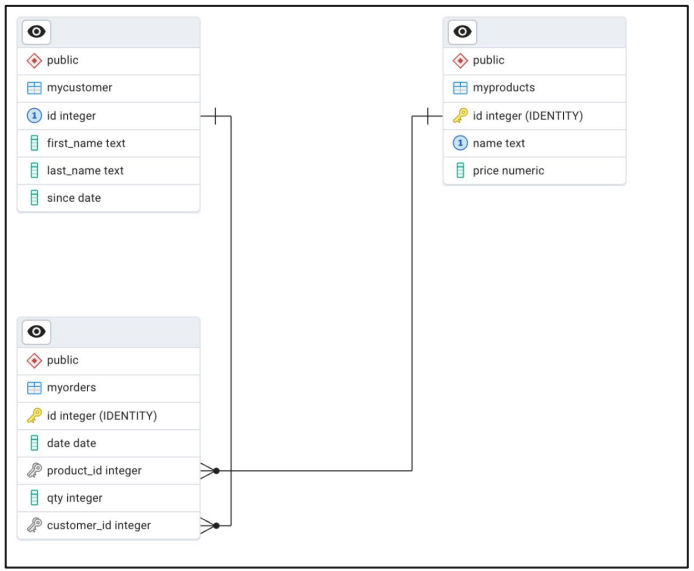
- Tables
- Columns
- Constraints
- Relationships
  - 1:1
  - 1:N
  - N:M



Two Notations



Chen Notation



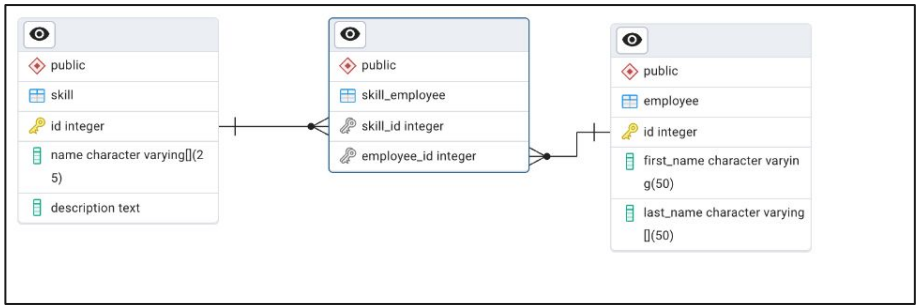
Crow Foot Notation

Different database



Use ERD Diagramming to create simple database

- Make sure you are connected to database DLH2
- Use pgAdmin ERD tool to define tables
  - Employees
  - Skills
  - Employees that have skill sets
- Use ERD tool to generate SQL
- Execute the generated SQL
- Inspect the results using the tree browser




public
skill
id integer
name character varying[](25)
description text
public
skill_employee
skill_id integer
employee_id integer
public
employee
id integer
first_name character varying(50)
last_name character varying[](50)

Data Normalization

Normalization and Avoiding Redundancy

- Data Normalization, a.k.a. 1NF/2NF,3NF is key to relational database design
- Data Normalization concepts do not exist in non-relational databases
    - Make sure every column only has one data element of the same type
    - Avoid redundancy (same data in multiple places)
    - Eliminate unintentional dependencies

Database Normal Forms

- **1st Normal Form (1NF)**: Tables do not include other tables; every column has one data element of the same type
- **2nd Normal Form (2NF)**: All data elements depend on the whole primary key
- **3rd Normal Form (3NF)**: No internal dependencies between data elements

Other (higher) normal forms exist, but are rarely applied in practice

Example

Article	Description	Brand	Country of origin	Sizes	Colors	Price
501 Jeans by Levis	The original button-fly jeans	Levis	USA	32/36 30/34 30/36	Blue	95.00
501 Jeans by Levis	The original button-fly jeans	Levis	USA	32/36 30/34 30/36	Black	110.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Blue	98.00
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Black	105.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Stone wash	101.99
Journey Trousers	Cotton gabardine Journey trousers	Brioni	Italy	36 38 40 42	Ivory White	125.95
Trucker Jacket	Rugged jeans jacket	Levis	USA	S M L	Blue	113.00

Example

Article	Description	Brand	Country of origin	Sizes		
501 Jeans by Levis	The original button-fly jeans	Levis	USA	32/36 30/34 30/36	Bl	95.00
501 Jeans by Levis	The original button-fly jeans	Levis	USA	32/36 30/34 30/36	Black	110.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Blue	98.00
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Black	105.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	32/36 30/34 30/36	Stone wash	101.99
Journey Trousers	Cotton gabardine Journey trousers	Brioni	Italy	36 38 40 42	Ivory White	125.95
Trucker Jacket	Rugged jeans jacket	Levis	USA	S M L	Blue	113.00

Sizes has multiple values in one column

Article	Description	Brand	Country of origin	Colors	Price
501 Jeans by Levis	The original button-fly jeans	Levis	USA	Blue	95.00
501 Jeans by Levis	The original button-fly jeans	Levis	USA	Black	110.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	Blue	98.00
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	Black	105.99
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA	Stone wash	101.99
Journey Trousers	Cotton gabardine Journey trousers	Brioni	Italy	Ivory White	125.95
Trucker Jacket	Rugged jeans jacket	Levis	USA	Blue	113.00

Article	Size
501 Jeans by Levis	32/36
501 Jeans by Levis	30/34
501 Jeans by Levis	30/36

To transform this into 1NF, we need to move the content of column Sizes to a separate table.

2NF: All data elements depend on the whole primary key

To meet the requirements of 2NF, where all attributes must depend on the *whole* primary key, we must address the following:

1. Brand depends on Article, that is, Article is the primary key for the content in column Brand
2. Article + Color are the combined primary key for Price

This leads us to create a separate table that links Article and Color to Price

2NF: Create a separate table that links Article and Color to Price

Article	Description	Brand	Country of origin
501 Jeans by Levis	The original button-fly jeans	Levis	USA
Wrangler Cowboy Cut	Western jeans for a great day	Wrangler	USA
Journey Trousers	Cotton gabardine Journey trousers	Brioni	Italy
Trucker Jacket	Rugged jeans jacket	Levis	USA

Article	Colors	Price
501 Jeans by Levis	Blue	95.00
501 Jeans by Levis	Black	110.99

3NF: No internal dependencies between data elements

Brand	Country of Origin
Levis	USA
Wrangler	USA
Brioni	Italy

Country of origin depends on brand (only). Move to separate table

3NF - No duplication; Easy access to data

Article	Description	Brand
501 Jeans by Levis	The original button-fly jeans	Levis
Wrangler Cowboy Cut	Pants	Wrangler
Journet Trousers	Pants	Brioni
Trucker Jacket	Outerwear	Levis

Brand	Country of Origin
Levis	USA
Wrangler	USA
Brioni	Italy

Article	Size
501 Jeans by Levis	32/36
501 Jeans by Levis	30/34
501 Jeans by Levis	30/36

Article	Color	Price
501 Jeans by Levis	Blue	95.00
501 Jeans by Levis	Black	110.99

Normalization Matters!

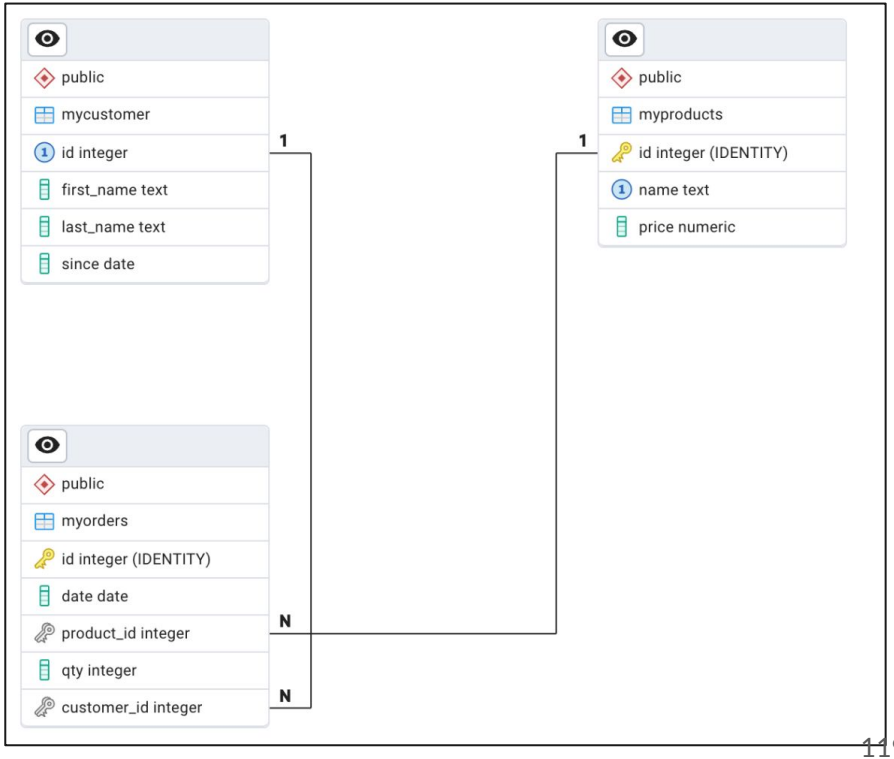
- Avoids managing the same data in multiple places
- Avoids strange deletion situations, e.g., brand disappears when no article uses it
- Makes sure there is always a way to refer to data in a unique way
- Avoids designing the process into the data

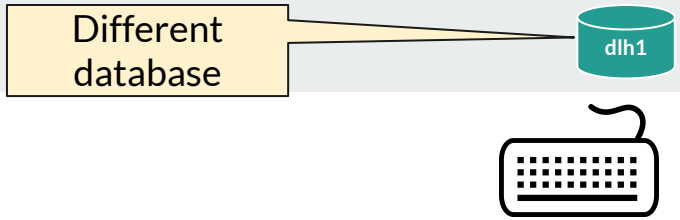
Joins

‘JOIN … ON …’ connects Two or more tables

Types of joins:

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN





Customers who ordered something

```
SELECT * FROM myorder
     JOIN mycustomer
     ON myorder.customer_id = mycustomer.id;
```

```
 id |    date    | product_nbr | qty | customer_id | id |        email         | first_name | last_name |   since
----+------------+-------------+-----+-------------+----+----------------------+------------+-----------+------------
  1 | 2025-02-02 | chisel_004  |   9 |           1 |  1 | marc@marclinster.com | Marc       | Linster   | 2025-01-01
```



- N.B.:
  - Explicit JOIN/ON improves readability
  - Using table alias (o,c) improves readability and avoids errors

```
SELECT
     o.id AS order_id,
     o.date AS order_date,
     o.qty AS order_quantity,
     c.first_name || ' ' || c.last_name AS customer_name
FROM myorder o
     JOIN mycustomer c
     ON o.customer_id = c.id;

 order_id | order_date | order_quantity | customer_name
----------+------------+----------------+---------------
        1 | 2025-02-02 |              9 | Marc Linster
```

How about the customers without orders?



LEFT OUTER JOIN

LEFT (OUTER) JOIN:

- All records from the first (left) table
- Joined with second (right) table if available ; show NULL (empty) columns otherwise
- ‘OUTER’ is optional

```
SELECT * FROM mycustomer c
      LEFT OUTER JOIN myorder o
      ON c.id = o.customer_id;
```

id	email	first_name	last_name	since	id	date	product_nbr	qty	customer_id
1	marc@marclinster.com	Marc	Linster	2025-01-01	1	2025-02-02	chisel_004	9	1
2	jmu@gmail.lu	Jeff	Mueller	2025-01-02					
3	bini@hotmail.lu	Jean	Bintner	2025-01-03					

Explaining Left Join

mycustomer (left of join)

id	email	first_	last_	since
1	marc@marclinster.com	Marc	Linster	1/1/2025
2	jmu@gmail.lu	Jeff	Mueller	1/2/2025
3	bini@hotmail.lu	Jean	Bintner	1/3/2025

myorder (right of join)

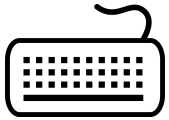
id	date	product_nbr	qty	customer_id
1	2/2/2025	chisel_004	9	1

In simple join:

- align customers and orders using mycustomer.id = myorder.customer_id
- List only the customers that have a matching order, and the orders that have a matching customer

In left join:

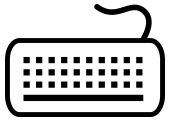
- align customers and orders using mycustomer.id = myorder.customer_id
- List all the customers (left) and if there is a matching order, list it too



Joining more tables: Customer with orders and product details

```
SELECT
     c.id, c.first_name, c.last_name,
     o.date, o.qty,
     p.name, p.price FROM mycustomer c
JOIN myorder o ON c.id = o.customer_id
JOIN myproduct p ON o.product_nbr = p.nbr;
```

```
 id | first_name | last_name |    date    | qty |     name      | price
----+------------+-----------+------------+-----+---------------+-------
  1 | Marc       | Linster   | 2025-02-02 |   9 | Chisel stone | 39.95
```



All customers, with orders and products (if they ordered anything)

```
SELECT c.id, c.first_name, c.last_name, o.id, o.date, o.qty, p.name, p.price FROM
mycustomer c
     LEFT JOIN myorder o ON c.id = o.customer_id
     LEFT JOIN myproduct p ON o.product_nbr = p.nbr;
```

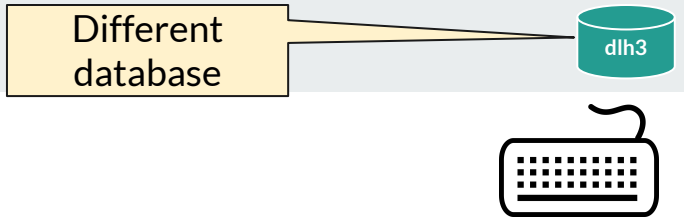
```
 id | first_name | last_name | id |    date    | qty |     name     | price
----+------------+-----------+----+------------+-----+--------------+-------
  1 | Marc       | Linster   |  1 | 2025-02-02 |   9 | Chisel stone | 39.95
  2 | Jeff       | Mueller   |    |            |     |              |
  3 | Jean       | Bintner   |    |            |     |              |
```

- See the importance of meaningful alias
- 'id' appears multiple times.
- customer_id, order_id, product_id would improve readability and reduce errors

Try It Out

- Find the customers who didn't order anything.
  - Show only the name and since when they are customers
- Find the products that were never ordered

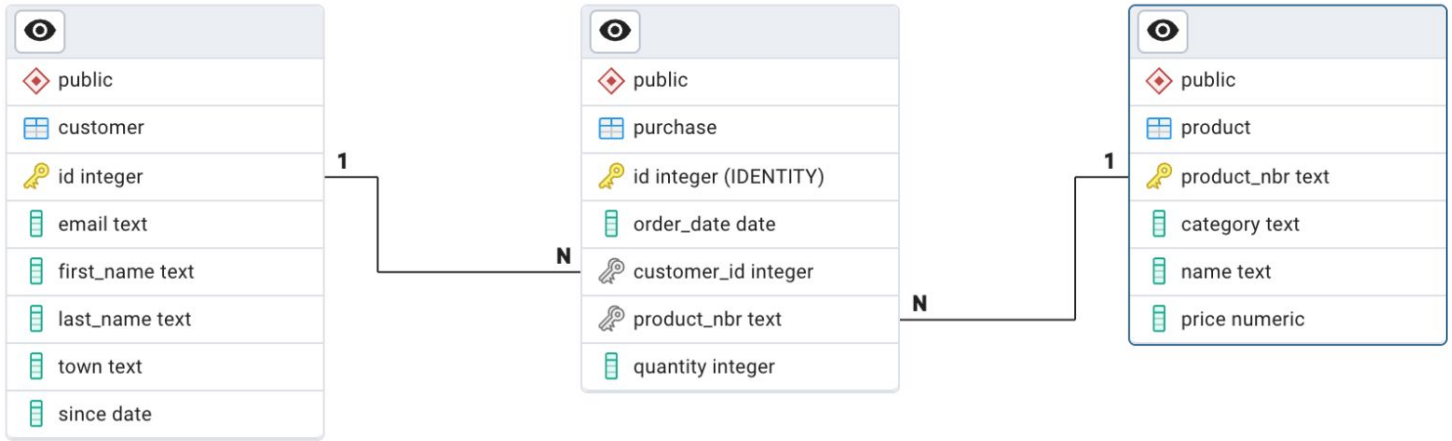
Views



Explore the data set

- Connect to database DLH3
- Create the ERD Diagram
- Explore the tables
  - SELECT * FROM ...;
  - SELECT COUNT(*) FROM ...;
- What is there?
- How many rows?

ERD Diagram

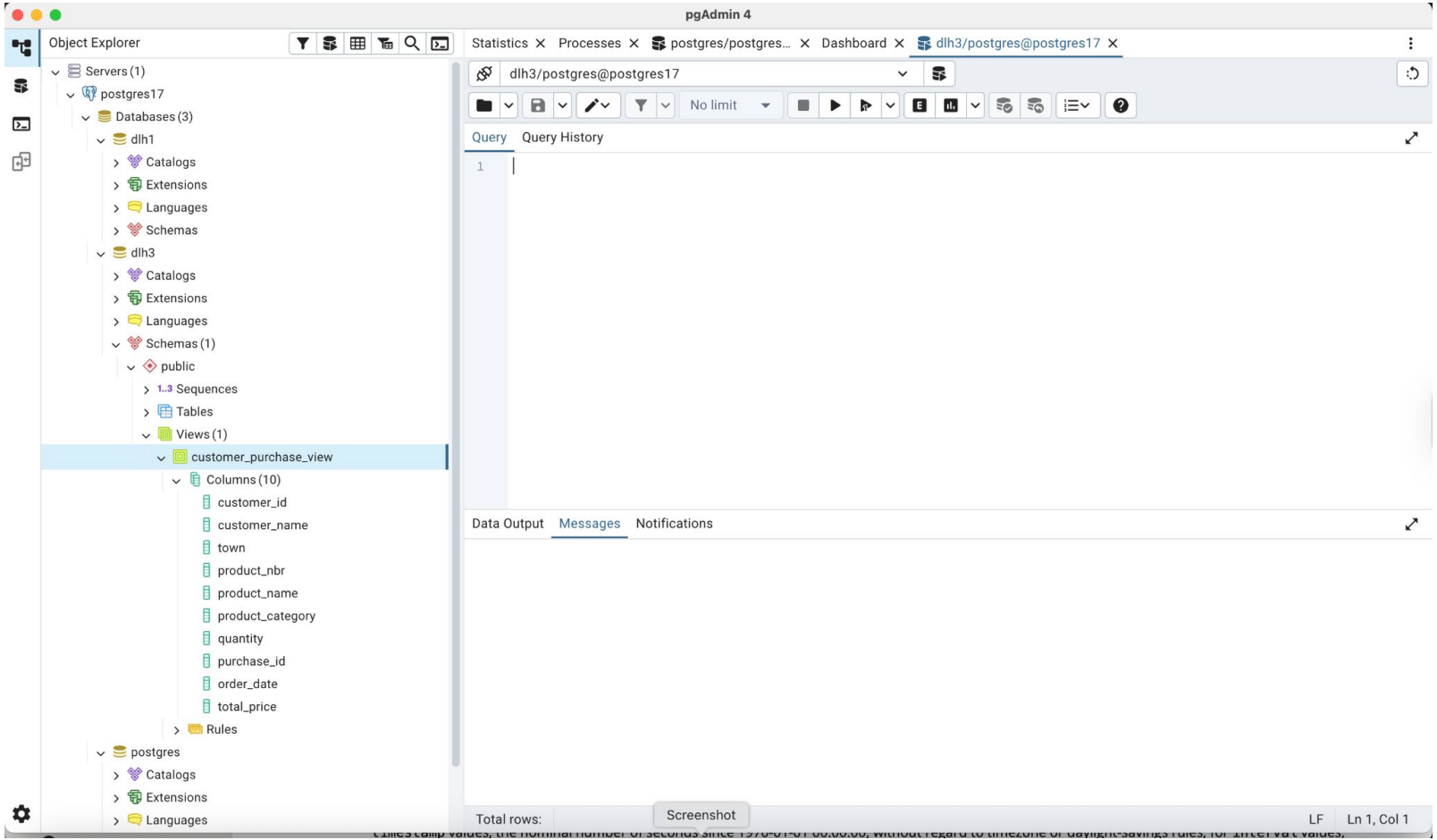

public
customer
id integer
email text
first_name text
last_name text
town text
since date
1
N
public
purchase
id integer (IDENTITY)
order_date date
customer_id integer
product_nbr text
quantity integer
N
1
public
product
product_nbr text
category text
name text
price numeric

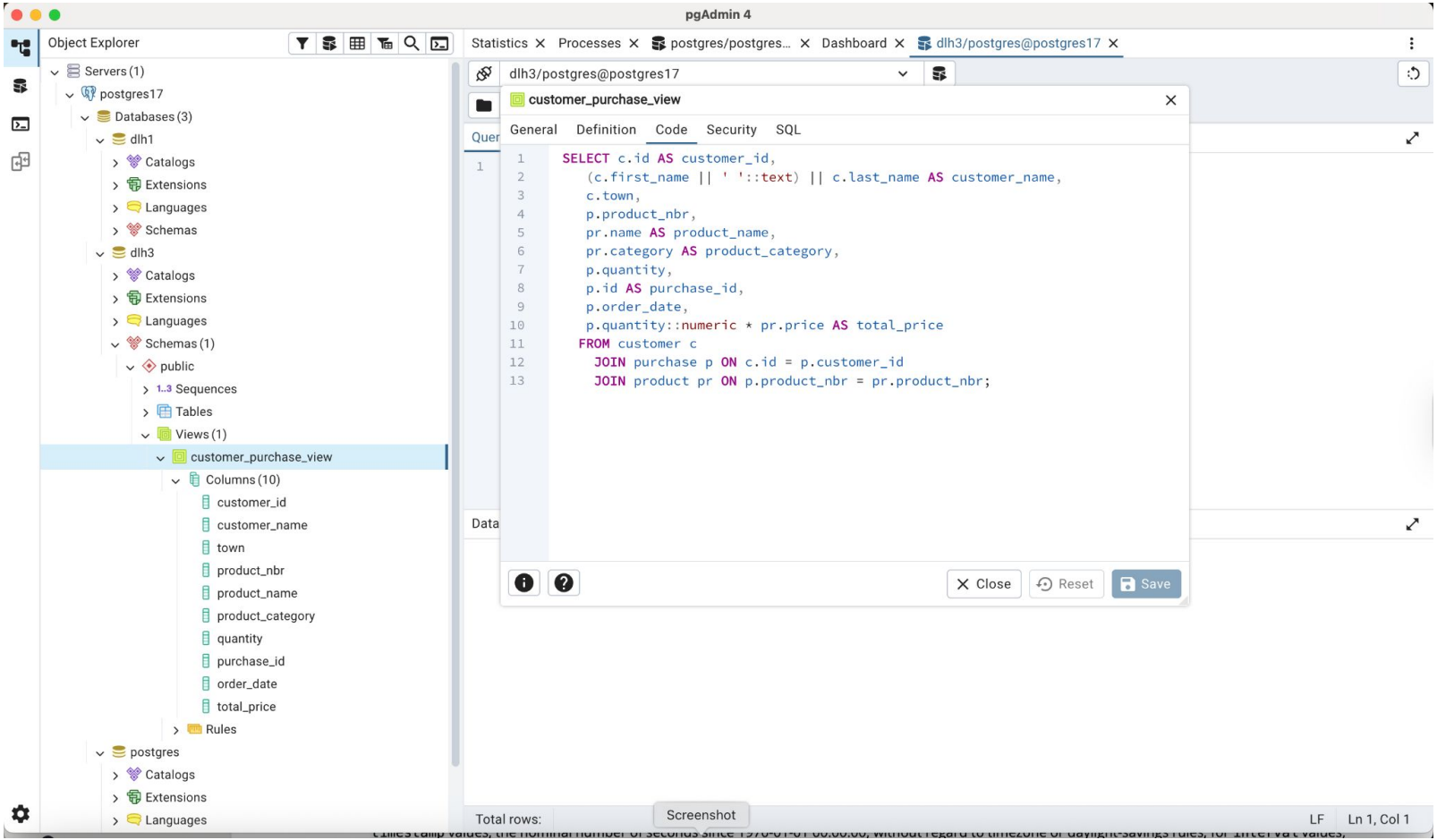
CREATE VIEW

Views are named queries that are stored in the database server

Try pgAdmin GUI and Query Tool

```
DROP VIEW IF EXISTS customer_purchase_view;
CREATE VIEW customer_purchase_view AS
    SELECT
        c.id AS customer_id,
        c.first_name || ' ' || c.last_name AS customer_name,
        c.town,
        p.product_nbr,
        pr.name AS product_name,
        pr.category as product_category,
        p.quantity,
        p.id AS purchase_id,
        p.order_date,
        (p.quantity * pr.price) AS total_price
    FROM customer c
    JOIN purchase p ON c.id = p.customer_id
    JOIN product pr ON p.product_nbr = pr.product_nbr;
```


pgAdmin 4
Object Explorer
Servers (1)
postgres17
Databases (3)
dlh1
Catalogs
Extensions
Languages
Schemas
dlh3
Catalogs
Extensions
Languages
Schemas (1)
public
Sequences
Tables
Views (1)
customer_purchase_view
Columns (10)
customer_id
customer_name
town
product_nbr
product_name
product_category
quantity
purchase_id
order_date
total_price
Rules
postgres
Catalogs
Extensions
Languages
Statistics
Processes
postgres/postgres...
Dashboard
dlh3/postgres@postgres17
dlh3/postgres@postgres17
No limit
Query
Query History
1
Data Output
Messages
Notifications
Total rows:
Screenshot
LF
Ln 1, Col 1


pgAdmin 4
Object Explorer
Servers (1)
postgres17
Databases (3)
dlh1
Catalogs
Extensions
Languages
Schemas
dlh3
Catalogs
Extensions
Languages
Schemas (1)
public
Sequences
Tables
Views (1)
customer_purchase_view
Columns (10)
customer_id
customer_name
town
product_nbr
product_name
product_category
quantity
purchase_id
order_date
total_price
Rules
postgres
Catalogs
Extensions
Languages
Statistics
Processes
postgres/postgres...
Dashboard
dlh3/postgres@postgres17
dlh3/postgres@postgres17
customer_purchase_view
General
Definition
Code
Security
SQL
SELECT c.id AS customer_id,
(c.first_name || ' '::text) || c.last_name AS customer_name,
c.town,
p.product_nbr,
pr.name AS product_name,
pr.category AS product_category,
p.quantity,
p.id AS purchase_id,
p.order_date,
p.quantity::numeric * pr.price AS total_price
FROM customer c
JOIN purchase p ON c.id = p.customer_id
JOIN product pr ON p.product_nbr = pr.product_nbr;
Close
Reset
Save
Total rows:
Screenshot
LF
Ln 1, Col 1

Call the VIEW

```
SELECT * FROM customer_purchase_view
   WHERE town = 'Esch'
   ORDER BY customer_name ASC, order_date DESC
   LIMIT 20;
```

Nota Bene

- VIEWS run the underlying queries every time they are accessed.
- VIEWS make it easy to share the same definitions within a team - i.e., every analyst uses the same definitions
- VIEWS are convenient and reduce code, but they do not significantly increase performance
- MATERIALIZED VIEWS are like VIEWS, but they create a table behind the scenes
- MATERIALIZED VIEWS pre-calculate the results
- MATERIALIZED VIEWS are
    - Much, much faster
    - Can (must) have their own indexes
    - Must be refreshed when the underlying data changes (In Postgres; Oracle has an incremental refresh)
    - Can be refreshed concurrently (no locking - needs a primary key) or non-concurrently (with locking - no PK needed)
- MATERIALIZED VIEWS are not in scope of the introductory course

Day 4

- Comments in SQL files
- Queries revisited: aggregates
- Indexes
- Writing good queries
- A sample project
- Recap
- Resources

Recap

Building Blocks of the simple SELECT Query

- SELECT
  - Columns to be selected, maybe using AS to provide a meaningful name
  - For example `last_name AS ln`
- FROM
  - Table(s) from which to select. Can also use AS to provide a new name
  - For example `customer AS c`
- WHERE
  - Filter clauses to select a subset of records
  - Can use the new names from the AS clause
  - For example `c.ln LIKE 'Li%'`
- ORDER BY
  - Organizes the output of the SELECT clause
  - Can use the new names from the AS clause
  - For example: `c.ln ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later

Postgres Operators in the WHERE Clause

`=`	Equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to
`<>`	Not equal to
`!=`	Not equal to
`LIKE`	Check if a value matches a pattern (case sensitive)
`ILIKE`	Check if a value matches a pattern (case insensitive)
`AND`	Logical AND
`OR`	Logical OR
`IN`	Check if a value is between a range of values
`BETWEEN`	Check if a value is between a range of values
`IS NULL`	Check if a value is NULL
`NOT`	Makes a negative result e.g. `NOT LIKE`, `NOT IN`, `NOT BETWEEN`

Working with strings in the SELECT clause

1) Appending strings

```
SELECT last_name || ', ' || first_name AS customer
     FROM mycustomer;
```

2) Formatting strings

```
SELECT
     FORMAT ('%s %s has been a valued customer since %s',
               first_name, last_name, since)
     FROM mycustomer;
```

3) Substrings

```
SELECT FORMAT ('%s. %s has been a valued customer since %s',
               SUBSTRING (first_name, 1,1), last_name, since)
     FROM mycustomer;
```

Working with numbers in the SELECT clause

1) Simple arithmetic

```
SELECT 15 * 4;
SELECT 15 / 4;
SELECT 15 / 4.0;
SELECT MOD (15, 4);
```

2) Casting

```
SELECT PI();
SELECT PI()::INTEGER;
SELECT PI()::NUMERIC;
```

3) Rounding

```
SELECT ROUND(PI()::NUMERIC);
SELECT ROUND(PI()::NUMERIC, 2);
```

Working with dates in the SELECT clause

1) Extracting date elements

```
SELECT last_name, since, EXTRACT (DAY FROM since) FROM mycustomer;
SELECT last_name, since, EXTRACT (YEAR FROM since) FROM mycustomer;
SELECT last_name, since, EXTRACT (DOW FROM since) FROM mycustomer;
```

2) Differences

```
SELECT last_name, since,  NOW()::date - since FROM mycustomer;
SELECT last_name, since, AGE( NOW(), since) FROM mycustomer;
SELECT last_name, since,
      EXTRACT (YEAR FROM AGE(NOW(), since)) AS nbr_years
      FROM mycustomer;
```

Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price > 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
  - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
  - You can assign a ‘trigger’ to handle deletions

Relations and foreign keys

```
DROP TABLE IF EXISTS myorder;

CREATE TABLE myorder (
      id INT
            PRIMARY KEY
            GENERATED BY DEFAULT AS IDENTITY,
      date DATE,
      customer_id INTEGER
            NOT NULL
            REFERENCES mycustomer (id),
      product_nbr VARCHAR(10)
            NOT NULL
            REFERENCES myproduct (nbr),
      qty INTEGER NOT NULL CHECK (qty > 0)
);
```

Is this clear?

try to create an order for a non-existing product

```
INSERT INTO myorder
      (date,customer_id, product_nbr, qty)
      VALUES ('2.2.2025', 1, 'cheese_10', 9);
;
```

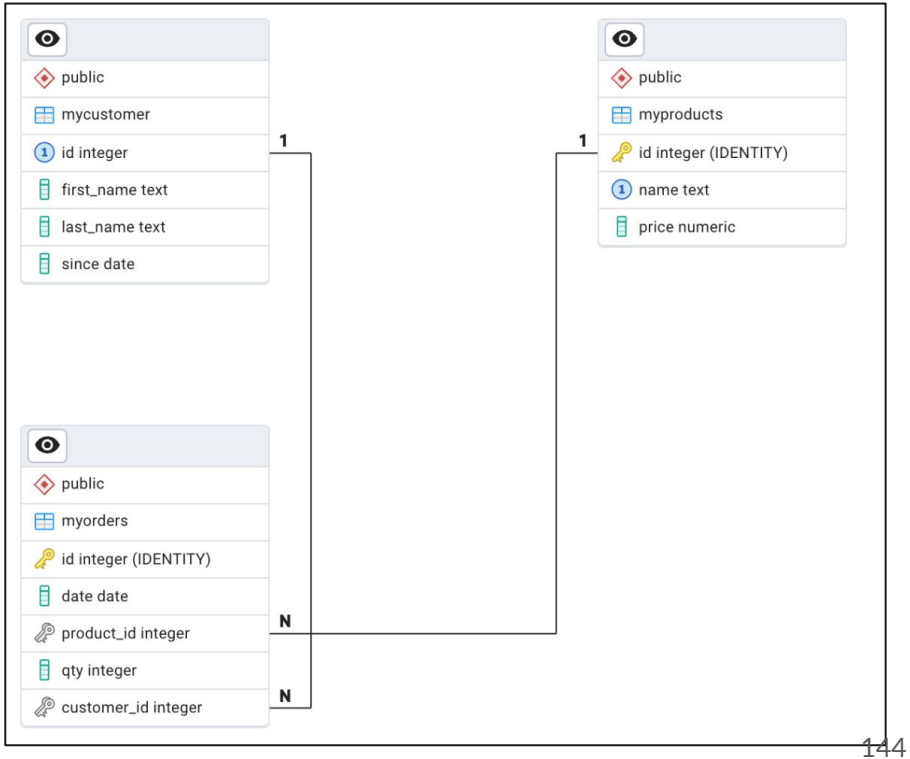
```
ERROR:  insert or update on table "myorder"
violates foreign key constraint
"myorder product nbr fkey"
Key (product_nbr)=(cheese_10) is not present
in table "myproduct".

SQL state: 23503
Detail: Key (product_nbr)=(cheese_10) is not
present in table "myproduct".
```

‘JOIN … ON …’ connects Two or more tables

Types of joins:

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN





LEFT OUTER JOIN

LEFT (OUTER) JOIN:

- All records from the first (left) table
- Joined with second (right) table if available ; show NULL (empty) columns otherwise
- ‘OUTER’ is optional

```
SELECT * FROM mycustomer c
      LEFT OUTER JOIN myorder o
      ON c.id = o.customer_id;
```

id	email	first_name	last_name	since	id	date	product_nbr	qty	customer_id
1	marc@marclinster.com	Marc	Linster	2025-01-01	1	2025-02-02	chisel_004	9	1
2	jmu@gmail.lu	Jeff	Mueller	2025-01-02					
3	bini@hotmail.lu	Jean	Bintner	2025-01-03					

Explaining Left Join

mycustomer (left of join)

id	email	first_	last_	since
1	marc@marclinster.com	Marc	Linster	1/1/2025
2	jmu@gmail.lu	Jeff	Mueller	1/2/2025
3	bini@hotmail.lu	Jean	Bintner	1/3/2025

myorder (right of join)

id	date	product_nbr	qty	customer_id
1	2/2/2025	chisel_004	9	1

In simple join:

- align customers and orders using mycustomer.id = myorder.customer_id
- List only the customers that have a matching order, and the orders that have a matching customer

In left join:

- align customers and orders using mycustomer.id = myorder.customer_id
- List all the customers (left) and if there is a matching order, list it too

Comments in SQL files

- '--' (two dashes) for a single-line comment
  - -- this is a single line comment. It should be less than 80 characters
- /* … */ for a multi-line comment
  - /* Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer ultricies, libero nec facilisis tristique, tortor ex accumsan nisi, nec placerat sem nunc ac urna. Cras enim leo, convallis sit amet purus sed, convallis ornare mi. Maecenas venenatis tempor sem, in ultricies libero mollis condimentum. Pellentesque suscipit porta quam at efficitur. Aliquam sit amet ipsum non ipsum malesuada iaculis. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur in consequat felis. Sed suscipit tristique ornare.
    */
- SQL comment:
  - `COMMENT ON TABLE myorder IS 'This is my order table';`

Aggregates

Questions that require aggregates

For example

- How many customers do we have in every town?
- Which town has the most customers?
- How much did the top 10 customers buy?

SQL Aggregates

- GROUP BY, e.g., group the customers by location
  - GROUP BY … HAVING allows to limit the groups

After the WHERE clause

- COUNT, SUM, AVG, MIN, MAX, e.g, the total/ average/ minimum/ maximum within a group

Part of the SELECT clause

Aggregates

```
SELECT group_by_element_1, group_by_element_2, …, aggregate_1, aggregate_2, …
     FROM table1
     JOIN table2 ON ….
     WHERE condition
     GROUP BY group_by_element_1, group_by_element_2, …;

SELECT COUNT (*) FROM customer;

SELECT town, COUNT(id)
     FROM customer
     GROUP BY TOWN
     ORDER BY town ASC;
```

Aggregates

```
SELECT town, COUNT(id) customer_count FROM customer
      GROUP BY TOWN
      ORDER BY town ASC;
```

Single group - one aggregate (COUNT)

```
SELECT town, COUNT(id) AS customer_count FROM customer
      WHERE since >= '2024-01-01' AND since < '2024-12-31'
      GROUP BY TOWN
      HAVING COUNT(id) > 1;
```

Single group - one aggregate; WHERE clause + limit of the groups

```
SELECT town, COUNT(id) AS customer_count FROM customer
      WHERE since >= '2024-01-01' AND since < '2024-12-31'
      GROUP BY TOWN
      HAVING COUNT(id) > 1
      ORDER BY customer_count DESC;
```

Single group - one aggregate; WHERE clause + limit of the groups + ORDER BY for groups

Aggregates

```
SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

Use the 'HAVING' clause to remove groups with small customer counts

```
SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
WHERE TOWN <> 'ESCH'
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

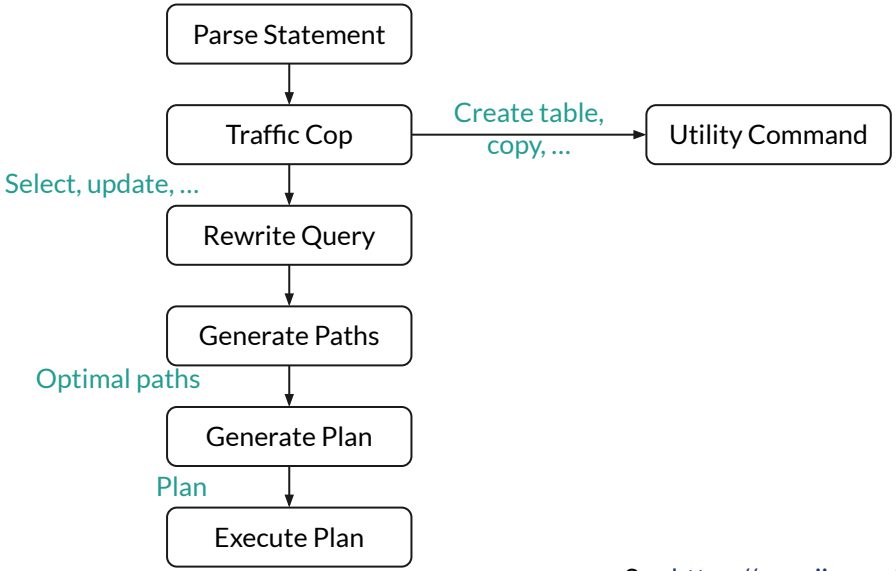
Use the 'HAVING' clause to remove groups with small customer counts
+
Use WHERE clause to filter on data set

Exercises

- How many food items did we sell in 2024
  - Join purchases and products
  - Filter by category food
  - Count(*)
- How many things did the top 10 customers buy?
  - Join the customer table with the purchase table
  - Group by customer and count the quantities
  - LIMIT 10
- How much did the top 10 customers spend every month?
  - Take a look at the view customer_purchase_view
  - Group by customer_name and make an aggregate of the total price
  - Order by price
  - LIMIT 10

Indexes

How does Postgres process a query?



See https://momjian.us/main/writings/pgsql/internalpics.pdf

How does a database find data?

- Every table is stored in 8k data blocks on disk, in files that are 1GB (or less)
- Example: `SELECT * from customer WHERE last_name = 'Voigt';`
- Without Index
  - Sequential scan of all data blocks for that table
  - If block is not in memory (shared buffers), get it from disk and load it into memory
    - “**Sequential Scan**”
- With Index
  - Load the index into memory (usually much smaller than data table)
  - Ask the index which blocks have the data
  - Load only those blocks
    - “**Index Scan**”

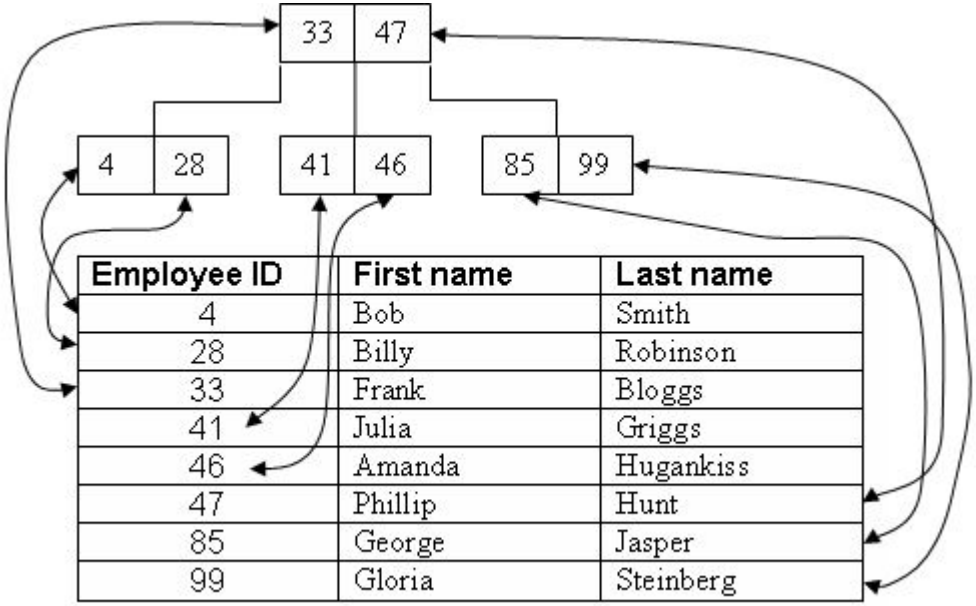
Many types of Indexes



- **Balanced Tree (B-Tree)**
  - **Every database has it. The most common and used for 90%+ of all cases. The default in Postgres**
  - **Ideal for looking up unique values and maintaining unique indexes**
  - **High concurrency implementation - very fast**
- Block-Range Index (BRIN)
  - Ideal for very large datasets that are organized according to the index
- Generalized Inverted Index (GIN)
  - Best for indexing values with many keys or values, e.g.,text documents, JSON, multi-dimensional data, arrays
  - Ideal for columns containing many duplicate
- Hash Index
  - Equality, non-equality lookups; no range lookups
- Generalized Search Tree (GiST)
  - Used for GIS, key-value store
- Space-Partitioned Generalized Search Tree (SP-GIST)
  - Specialized index

https://momjian.us/main/writings/pgsql/indexing.pdf

Very Fast Access to Data



Employee ID	First name	Last name
4	Bob	Smith
28	Billy	Robinson
33	Frank	Bloggs
41	Julia	Griggs
46	Amanda	Hugankiss
47	Phillip	Hunt
85	George	Jasper
99	Gloria	Steinberg

https://en.wikipedia.org/wiki/Database

Writing Good Queries

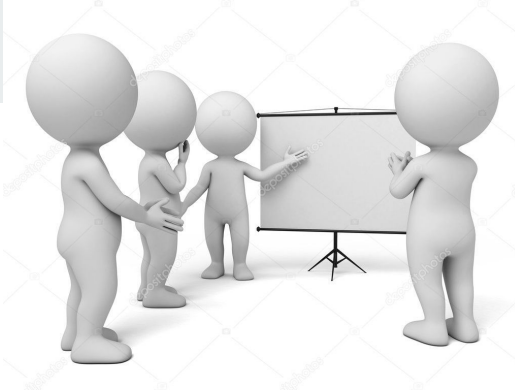
Summary: Keys to Writing Good Queries

1. Limit the number of rows that are returned
2. Limit the number of columns that are returned
3. Leverage indexes where available
4. Define aliases in the SELECT clause using ‘AS’ to avoid duplicate column names
5. Use the aliases in the WHERE and GROUP clause to avoid typos and shorten the code
6. Define highly selective WHERE clauses, especially when using JOINS
7. Avoid CROSS JOINS (data set explodes!)
8. Use VIEWS to standardize complex, often repeated queries (e.g., sales_transaction combined with sales_transaction_line)
9. Use snake_case names for tables and columns

SQL Order of Execution

1. **FROM/JOIN**: Specifies the tables from which to retrieve data.
2. **WHERE**: Filters the rows that meet the condition before grouping.
3. **GROUP BY**: Groups rows that share a property.
4. **HAVING**: Filters groups based on conditions, applied after grouping.
5. **SELECT**: Specifies the columns to retrieve or calculate.
6. **DISTINCT**: Removes duplicate rows from the result set.
7. **ORDER BY**: Sorts the result set by specified columns.
8. **LIMIT**: Specifies the maximum number of rows to return.
9. **OFFSET**: Specifies how many rows to skip before starting to return rows.

A sample project

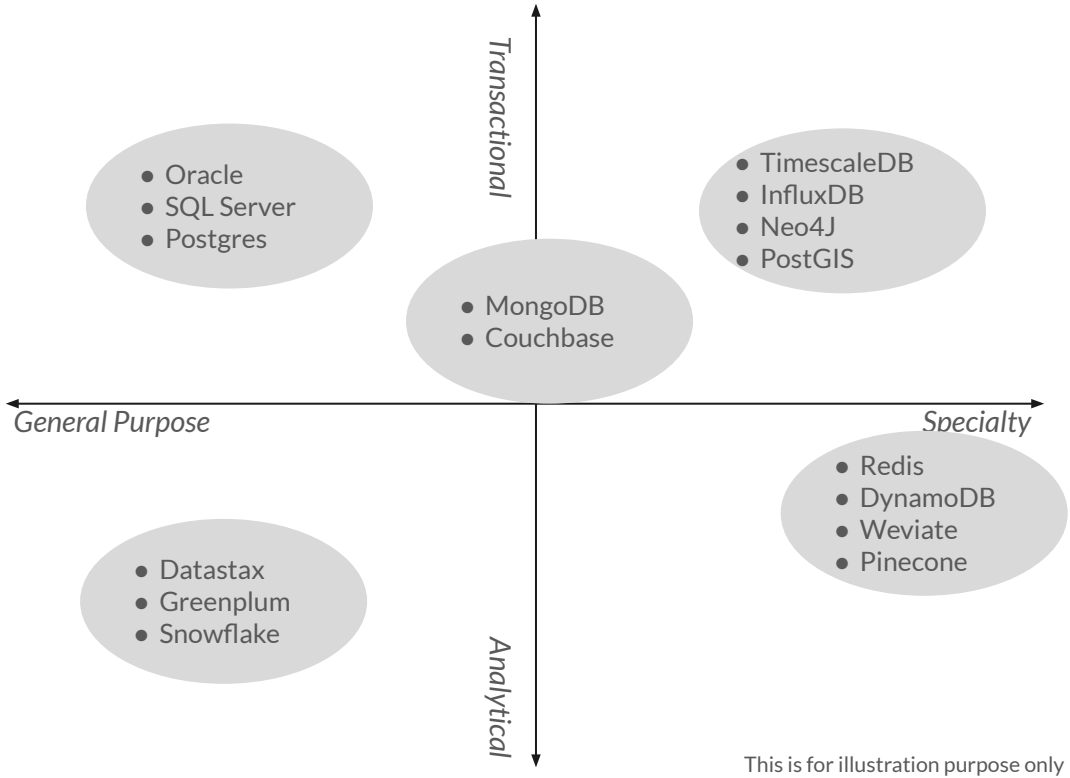


Project

- Build a database (DLHX) for a company with departments, employees and managers working in departments, projects, employee assignment to projects, and hours recorded for the employees working on the project
- Simple queries
  - Who worked on which project?
  - Who had the most hours total last month?
  - What were the total hours last month for each project?
  - Did we have any projects with 0 hours?

Recap

Database Landscape



This is for illustration purpose only
There are many, many databases and many ways to classify them

Table Basics

- Rows = records (a.k.a. tuples in Postgres)
- Columns = data elements
- Example: mycustomer table
  - ID = a way to uniquely identify a customer ⇒ usually a large number
  - Email = the email address, a text field, sometimes limited in length
  - First name ⇒ a text field, sometimes limited in length
  - Last name ⇒ a text field, sometimes limited in length
  - Since => a date field

```
CREATE TABLE mycustomer (
      id INTEGER,
      email TEXT,
      first_name TEXT,
      last_name TEXT,
      since DATE
);
```

```
postgres=# SELECT * FROM mycustomer;
 id |        email         | first_name | last_name |   since
----+----------------------+------------+-----------+------------
  1 | marc@marclinster.com | Marc       | Linster   | 2025-01-01
  2 | jmu@gmail.lu         | Jeff       | Mueller   | 2025-01-02
  3 | bini@hotmail.lu      | Jean       | Bintner   | 2025-01-03
```

Query types

- SELECT: List data from tables that meet certain conditions
- UPDATE: Change data in records that meet certain conditions
- INSERT: Add records
- DELETE: Remove Records

Building Blocks of the simple SELECT Query

- SELECT
  - Columns to be selected, maybe using AS to provide a meaningful name
  - For example `last_name AS ln`
- FROM
  - Table(s) from which to select. Can also use AS to provide a new name
  - For example `customer AS c`
- WHERE
  - Filter clauses to select a subset of records
  - Can use the new names from the AS clause
  - For example `c.last_name LIKE 'Li%'`
- ORDER BY
  - Organizes the output of the SELECT clause
  - Can use the new names from the AS clause
  - For example: `c.last_name ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later

Postgres Operators in the WHERE Clause

`=`	Equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to
`<>`	Not equal to
`!=`	Not equal to
`LIKE`	Check if a value matches a pattern (case sensitive)
`ILIKE`	Check if a value matches a pattern (case insensitive)
`AND`	Logical AND
`OR`	Logical OR
`IN`	Check if a value is between a range of values
`BETWEEN`	Check if a value is between a range of values
`IS NULL`	Check if a value is NULL
`NOT`	Makes a negative result e.g. `NOT LIKE`, `NOT IN`, `NOT BETWEEN`

Postgres functions in the SELECT clause

Some of the key string functions

||	String append	`'a'||'_'||'b' → a_b`
lpad/rpad	Create a padded string	`rpad('hi', 5, 'xy') → hixyx`
substring	Create a substring	`substring('Thomas' from 2 for 3) → hom`
::string	Cast a number (and other types) as string	`1::TEXT||'_'||2::TEXT → 1_2`

Many, many more in the Postgres documentation: https://www.postgresql.org/docs/current/functions-string.html

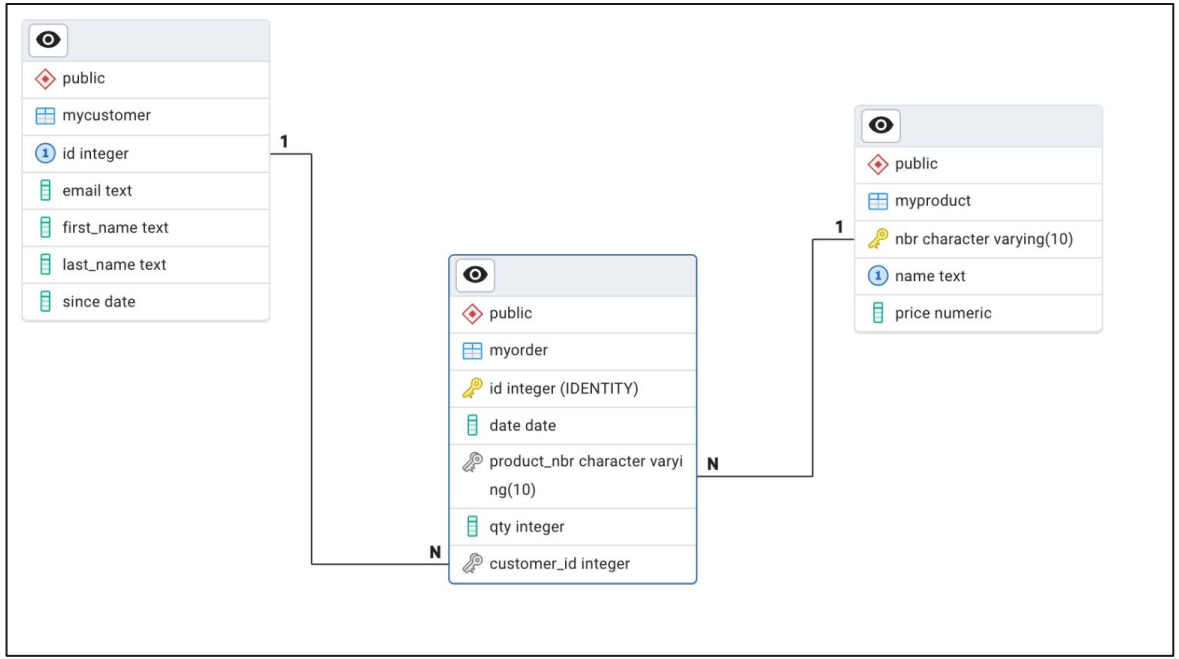
Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price > 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
  - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
  - You can assign a ‘trigger’ to handle deletions

Schemas

Graphical view of the schema

- Tables
- Columns
- Constraints
- Relationships
  - 1:1
  - 1:N
  - N:M



3NF - No duplication; Easy access to data

Article	Description	Brand
501 Jeans by Levis	The original button-fly jeans	Levis
Wrangler Cowboy Cut	Pants	Wrangler
Journet Trousers	Pants	Brioni
Trucker Jacket	Outerwear	Levis

Brand	Country of Origin
Levis	USA
Wrangler	USA
Brioni	Italy

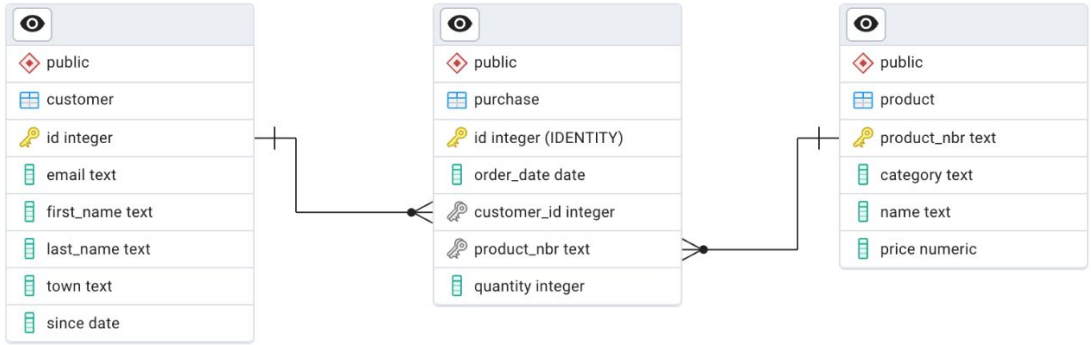
Article	Size
501 Jeans by Levis	32/36
501 Jeans by Levis	30/34
501 Jeans by Levis	30/36

Article	Color	Price
501 Jeans by Levis	Blue	95.00
501 Jeans by Levis	Black	110.99

‘JOIN … ON …’ connects Two or more tables

Types of joins:

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN



CREATE VIEW

Views are named queries that are stored in the database server

```
CREATE OR REPLACE VIEW mycustomer_with_purchase_view AS
    SELECT
        c.id AS customer_id,
        FORMAT ('%s %s', c.first_name, c.last_name)
            AS customer_name,
        c.since AS customer_since,
        o.id AS order_number,
        o.date AS order_date,
        p.name AS product_name,
        p.price AS product_unit_price,
        p.price * o.qty AS extended_price
    FROM mycustomer c
        LEFT JOIN myorder o ON c.id = o.customer_id
        LEFT JOIN myproduct p ON o.product_nbr = p.nbr;

SELECT * FROM mycustomer_with_purchase_view;
```

Try `\d` and `\d+ mycustomer_with_purchase_view`

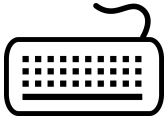
SQL Aggregates

- GROUP BY, e.g., group the customers by location
  - GROUP BY … HAVING allows to limit the groups

After the WHERE clause

- SUM, AVG, MIN, MAX, e.g, the total/ average/ minimum/ maximum within a group

Part of the SELECT clause



How many customers by town?

```
SELECT town, COUNT(id) customer_count FROM customer
      GROUP BY TOWN
      ORDER BY town ASC;

SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
WHERE TOWN <> 'ESCH'
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

Many types of Indexes



- Balanced Tree (B-Tree)
  - Every database has it. The most common and used for 90%+ of all cases. The default in Postgres
  - Ideal for looking up unique values and maintaining unique indexes
  - High concurrency implementation - very fast
- Block-Range Index (BRIN)
  - Ideal for very large datasets that are organized according to the index
- Generalized Inverted Index (GIN)
  - Best for indexing values with many keys or values, e.g.,text documents, JSON, multi-dimensional data, arrays
  - Ideal for columns containing many duplicate
- Hash Index
  - Equality, non-equality lookups; no range lookups
- Generalized Search Tree (GiST)
  - Used for GIS, key-value store
- Space-Partitioned Generalized Search Tree (SP-GIST)
  - Specialized index

https://momjian.us/main/writings/pgsql/indexing.pdf

Summary: Keys to Writing Good Queries

1. Limit the number of rows that are returned
2. Limit the number of columns that are returned
3. Leverage indexes where available
4. Define aliases in the SELECT clause using 'AS' to avoid duplicate column names
5. Use the aliases in the WHERE and GROUP clause to avoid typos and shorten the code
6. Define highly selective WHERE clauses, especially when using JOINS
7. Avoid CROSS JOINS (data set explodes!)
8. Use VIEWS to standardize complex, often repeated queries (e.g., sales_transaction combined with sales_transaction_line)
9. Use snake_case names for tables and columns

See Jimmy Angelakos' book: https://livebook.manning.com/book/postgresql-mistakes-and-how-to-avoid-them/welcome/v-9/

SQL Order of Execution

1. **FROM/JOIN**: Specifies the tables from which to retrieve data.
2. **WHERE**: Filters the rows that meet the condition before grouping.
3. **GROUP BY**: Groups rows that share a property.
4. **HAVING**: Filters groups based on conditions, applied after grouping.
5. **SELECT**: Specifies the columns to retrieve or calculate.
6. **DISTINCT**: Removes duplicate rows from the result set.
7. **ORDER BY**: Sorts the result set by specified columns.
8. **LIMIT**: Specifies the maximum number of rows to return.
9. **OFFSET**: Specifies how many rows to skip before starting to return rows.

Resources

Resources



Discount Code; MARC20

Books:

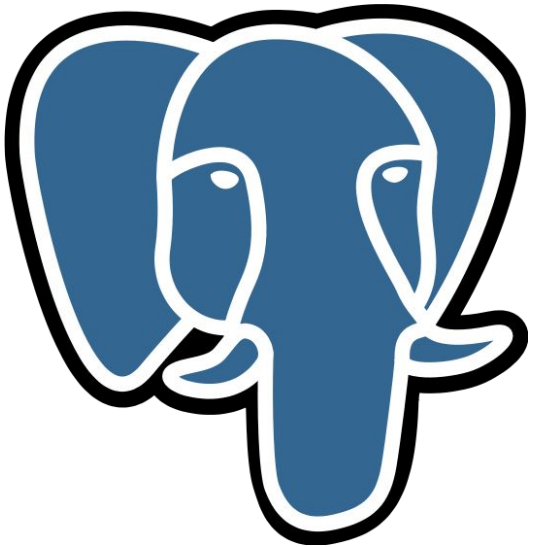
- AI-Ready PostgreSQL 18: Vibhor Kumar, Marc Linster 2026
- PostgreSQL 16 Administration Cookbook; Gianni Ciolli, Boris Mejias, Jimmy Angelakos, Vibhor Kumar, Simon Riggs, 2023
- PostgreSQL Up and Running; Regina Obe and Leo Hsu, 2020
- Introduction to PostgreSQL for the data professional; Ryan Booz and Grant Fritchey 2025 (coming from SQL Server)
- Mastering PostgreSQL 17: Elevate your database skills with advanced deployment, optimization, and security strategies 6th ed. Hans-Jürgen Schönig, 2023

Blogs & online:

- History of databases (video) and SQL (video)
- Postgres documentation: https://www.postgresql.org/docs/current/index.html
- Neon's Postgres Tutorial: https://neon.tech/postgresql/tutorial
- Bruce Momjian's blog about SQL and Postgres: https://momjian.us/main/presentations/sql.html
- Bruce Momjian's blog about Postgres performance: https://momjian.us/main/presentations/performance.html
- Laetitia Avrot's excellent blog about DBA topics: https://mydbanotebook.org/
- pgAdmin Website (with videos): https://www.pgadmin.org/

Course Feedback





Use Postgres - Get Stuff Done!